

eclipse와 git

학습목표

eclipse와 github 연동 방법을 학습한다

목차

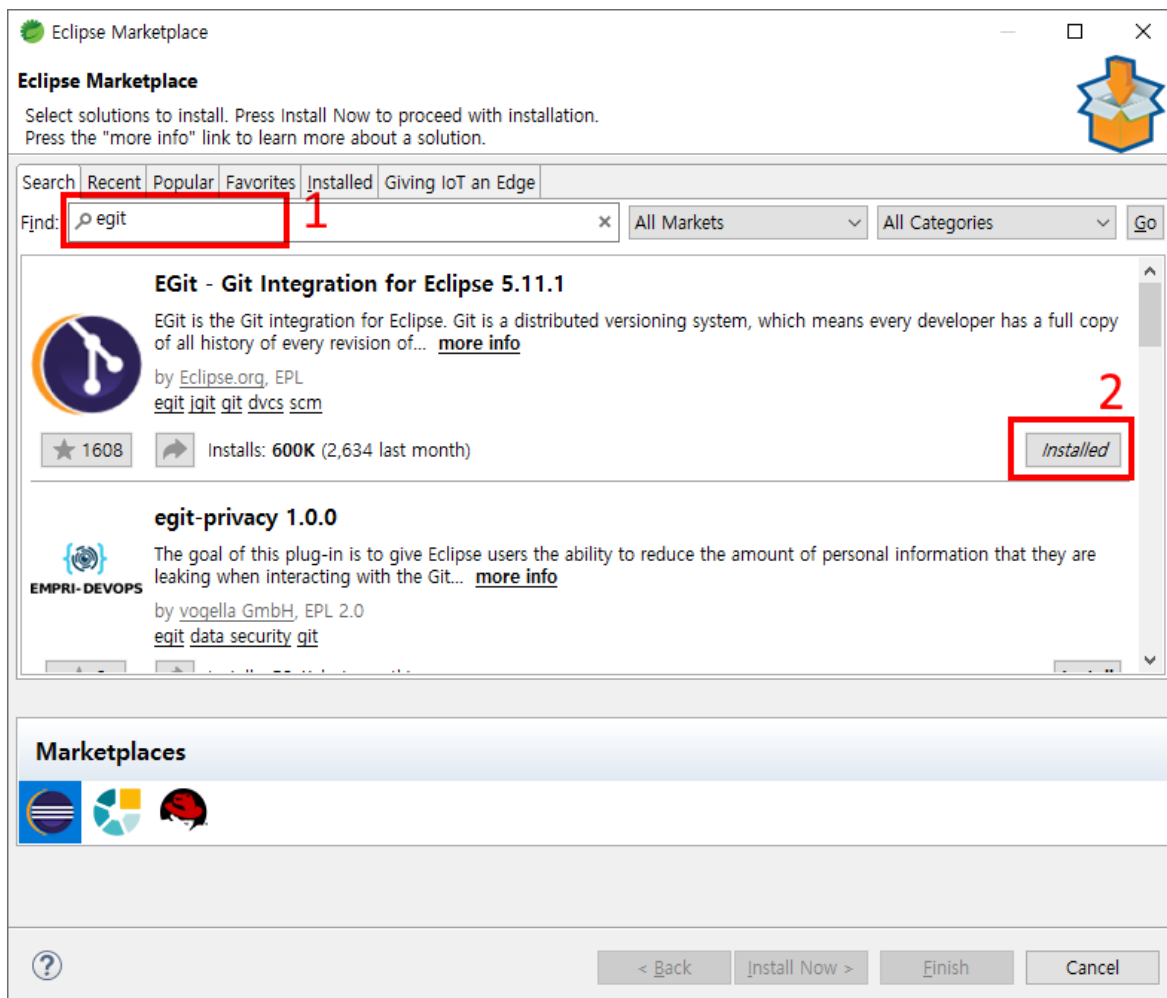
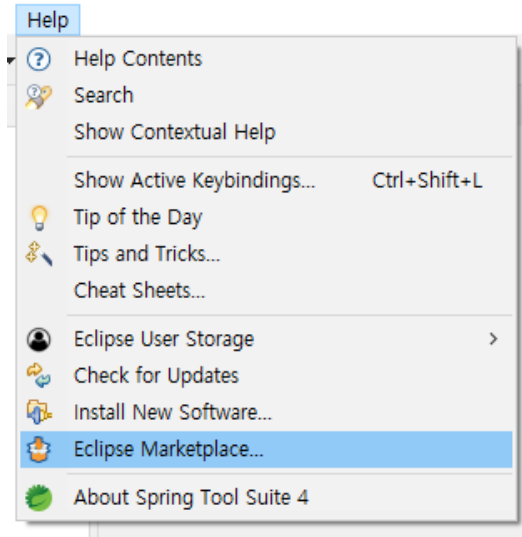
1. EGit 설치	2
1) EGit 설치	2
2. 프로젝트 생성	4
1) 프로젝트 생성.....	4
2) Java 클래스 생성.....	6
3) 프로젝트에 local repository 생성.....	8
4) 파일의 상태.....	10
5) commit	11
6) .gitignore 파일 생성.....	15
7) Hello.java 수정.....	19
8) 다시 commit.....	19
3. remote repository 생성.....	21
1) github에 repository 생성.....	21
2) repository URL 확인.....	22
4. remote repository에 push.....	23
1) 최초 push	23
2) push 결과 확인	24
5. 프로젝트 가져오기.....	25
1) git_test2 프로젝트 삭제.....	25
2) github에서 프로젝트 가져오기 #1.....	26
3) github에서 프로젝트 가져오기 #2.....	31
4) github에서 프로젝트 가져오기 #3.....	38
5) github에서 프로젝트 가져오기 #4.....	41
6. 과제	46

1. EGit 설치

eclipse에 git 클라이언트 도구를 설치하자.
이것을 EGit 이라고 부른다.

Spring Tools Suite도 eclipse 이다.
(eclipse + spring tools = spring tool suite)

1) EGit 설치



(1) egit 입력하고 엔터키

(2)

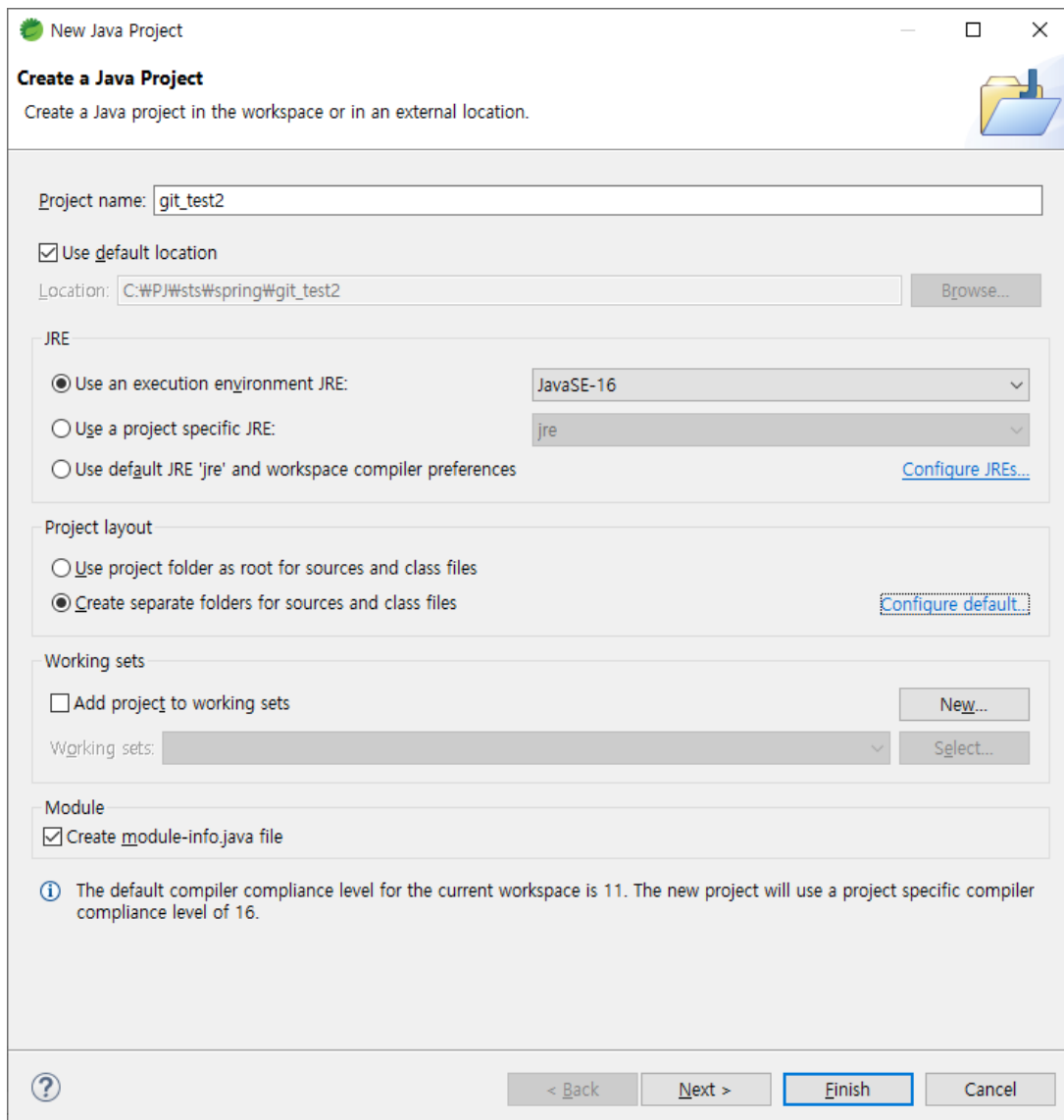
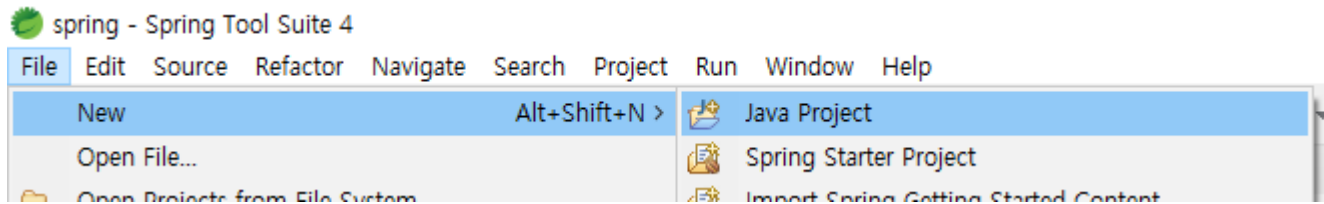
EGit 항목의 Install 버튼 클릭하여 설치하자.

만약 이 버튼의 텍스트가 Installed 이면 이미 EGit이 설치된 것이니 또 설치할 필요 없다.

2. 프로젝트 생성

아래 절차는 프로젝트를 github repository에 처음 등록할 때 필요한 작업이다.
프로젝트 팀장이 최초로 한 번만 수행하면 된다.

1) 프로젝트 생성



Project name: git_test2
Finish 클릭

New module-info.java

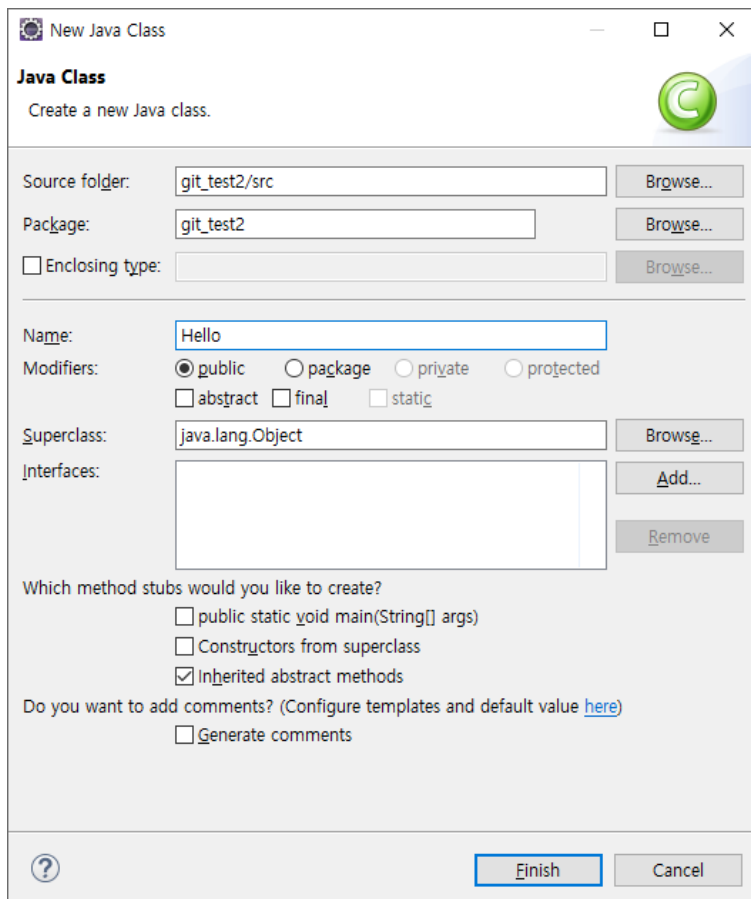
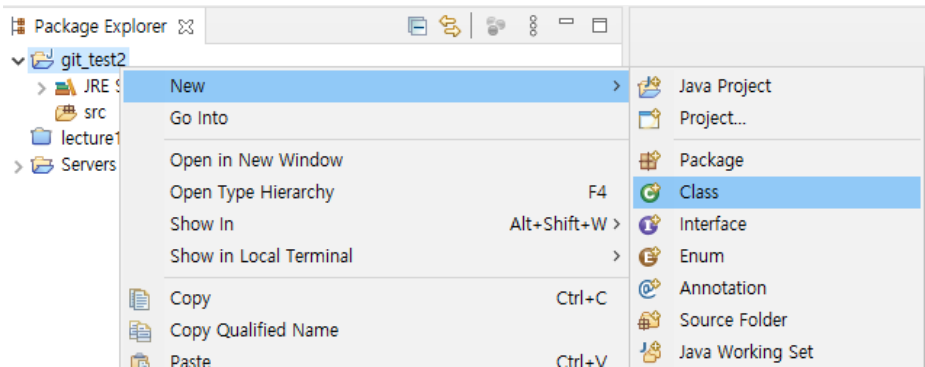
Create module-info.java
Create a new module-info.java file.

Module name:

☐ Generate comments (configure templates and default value [here](#))

Don't Create 클릭

2) Java 클래스 생성



Hello.java

```
package git_test2;

public class Hello {

    public void say() {
        System.out.println("hello world");
    }

}
```

Main.java

```
package git_test2;

public class Main {

    public static void main(String[] args) {
        new Hello().say();
    }

}
```

3) 프로젝트에 local repository 생성

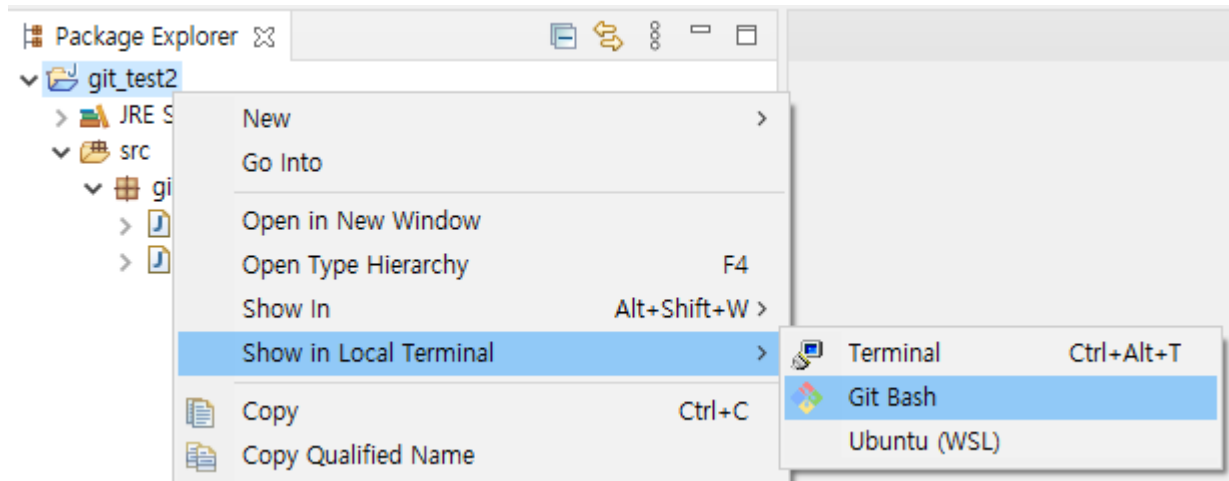
새 프로젝트의 경우 먼저 프로젝트에 local repository를 만들어야 한다.

이클립스에서도 local repository를 생성할 수 있지만,
그렇게 하면 기본 브랜치 이름이 master 되는 문제가 있다.

그래서 다음과 같은 절차로 local repository를 생성해야 한다.

- (1) git bash 셸에서 local repository를 생성
- (2) 이클립스에서 local repository 사용하기

git bash 셸 열기



git Bash 메뉴를 클릭하면,
터미널 창이 열리고, 현재 프로젝트 디렉토리에서 git bash 셸이 실행된다.



git_test2 프로젝트 디렉토리에서 git bash 셸이 실행중이다.

local repository 생성 (git init)

```
MINGW64:/c/PJ/STS/java/git_test2

Seungjin@ryzen5 MINGW64 /c/PJ/STS/java/git_test2
$ git init
Initialized empty Git repository in C:/PJ/STS/java/git_test2/.git/

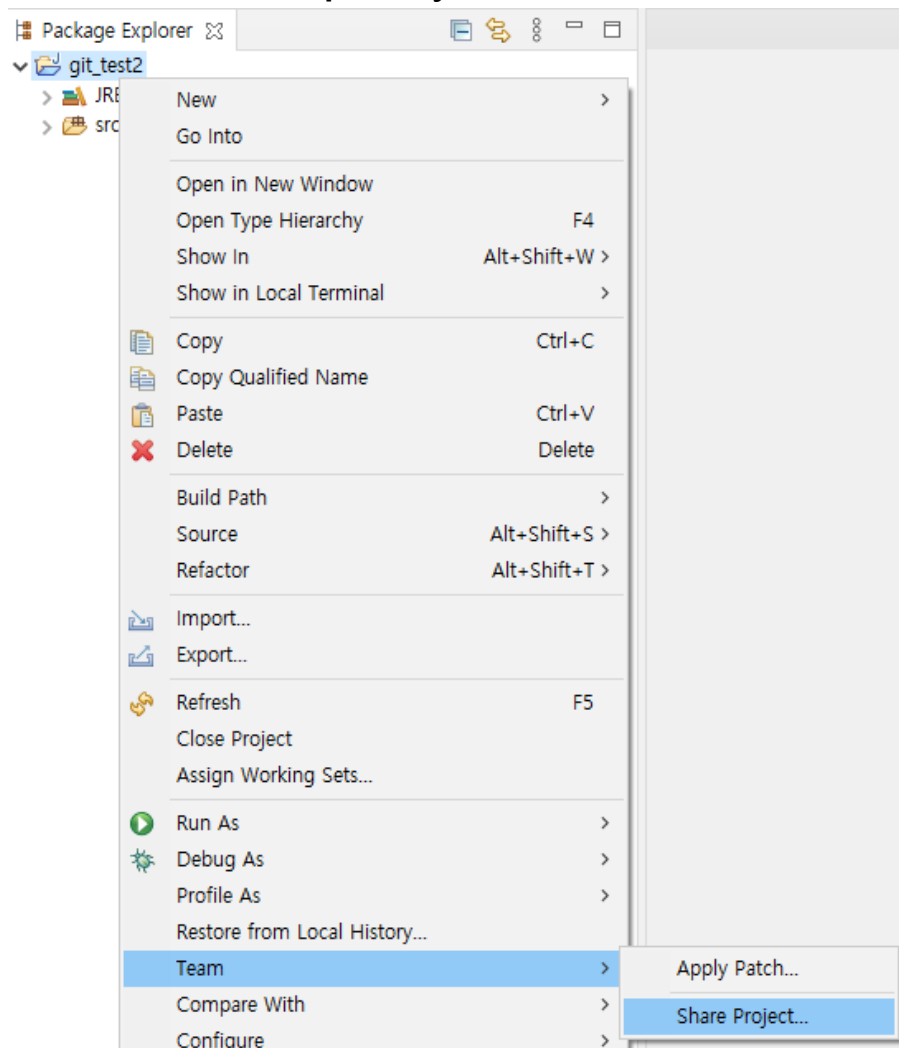
Seungjin@ryzen5 MINGW64 /c/PJ/STS/java/git_test2 (main)
$ ls -al
total 10
drwxr-xr-x 1 Seungjin 197609  0  9월  4 08:26 ./
drwxr-xr-x 1 Seungjin 197609  0  9월  4 07:50 ../
-rw-r--r-- 1 Seungjin 197609 386  9월  4 07:50 .classpath
drwxr-xr-x 1 Seungjin 197609  0  9월  4 08:26 .git/
-rw-r--r-- 1 Seungjin 197609 368  9월  4 07:50 .project
drwxr-xr-x 1 Seungjin 197609  0  9월  4 07:50 .settings/
drwxr-xr-x 1 Seungjin 197609  0  9월  4 07:51 bin/
drwxr-xr-x 1 Seungjin 197609  0  9월  4 07:51 src/

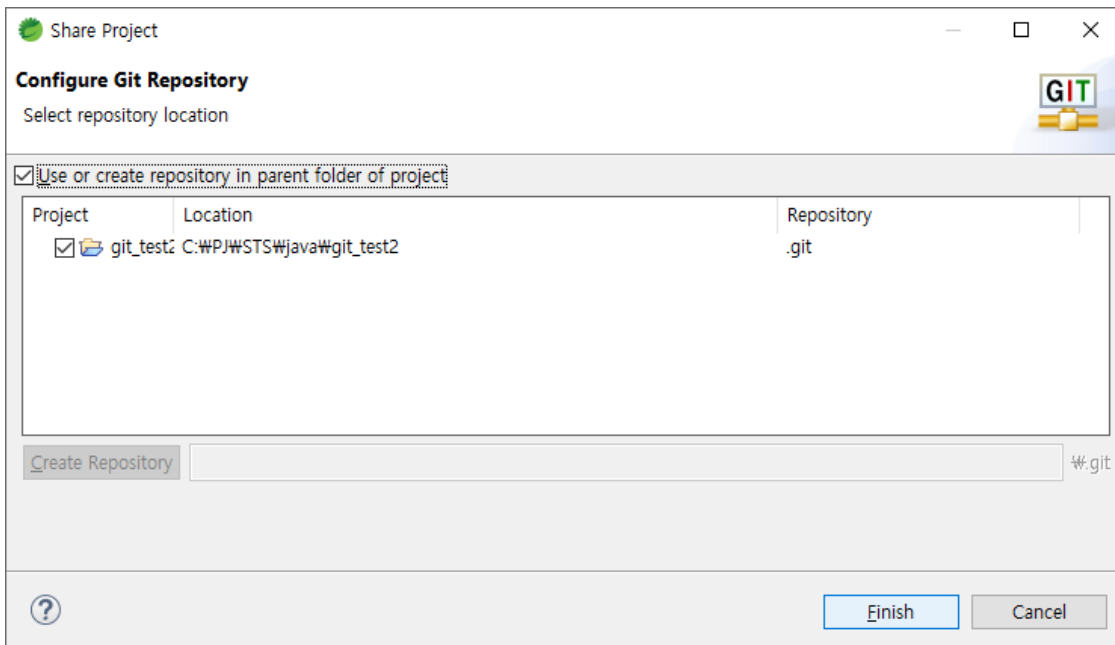
Seungjin@ryzen5 MINGW64 /c/PJ/STS/java/git_test2 (main)
$
```

git init 명령을 실행하면, 현재 디렉토리에 local repository가 생성한다.
ls -al 명령은 현재 디렉토리의 파일 목록을 출력한다.

.git 디렉토리가 생성되었다.
이 디렉토리가 local repository 이다.

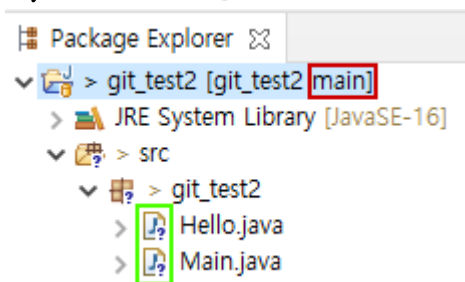
이클립스에서 local repository 사용하기





Finish 클릭

4) 파일의 상태



빨간색 사각형 부분

현재 브랜치 이름이 main 이다.

녹색 사각형 부분

파일 아이콘에 작은 물음표가 새로 생겼다.

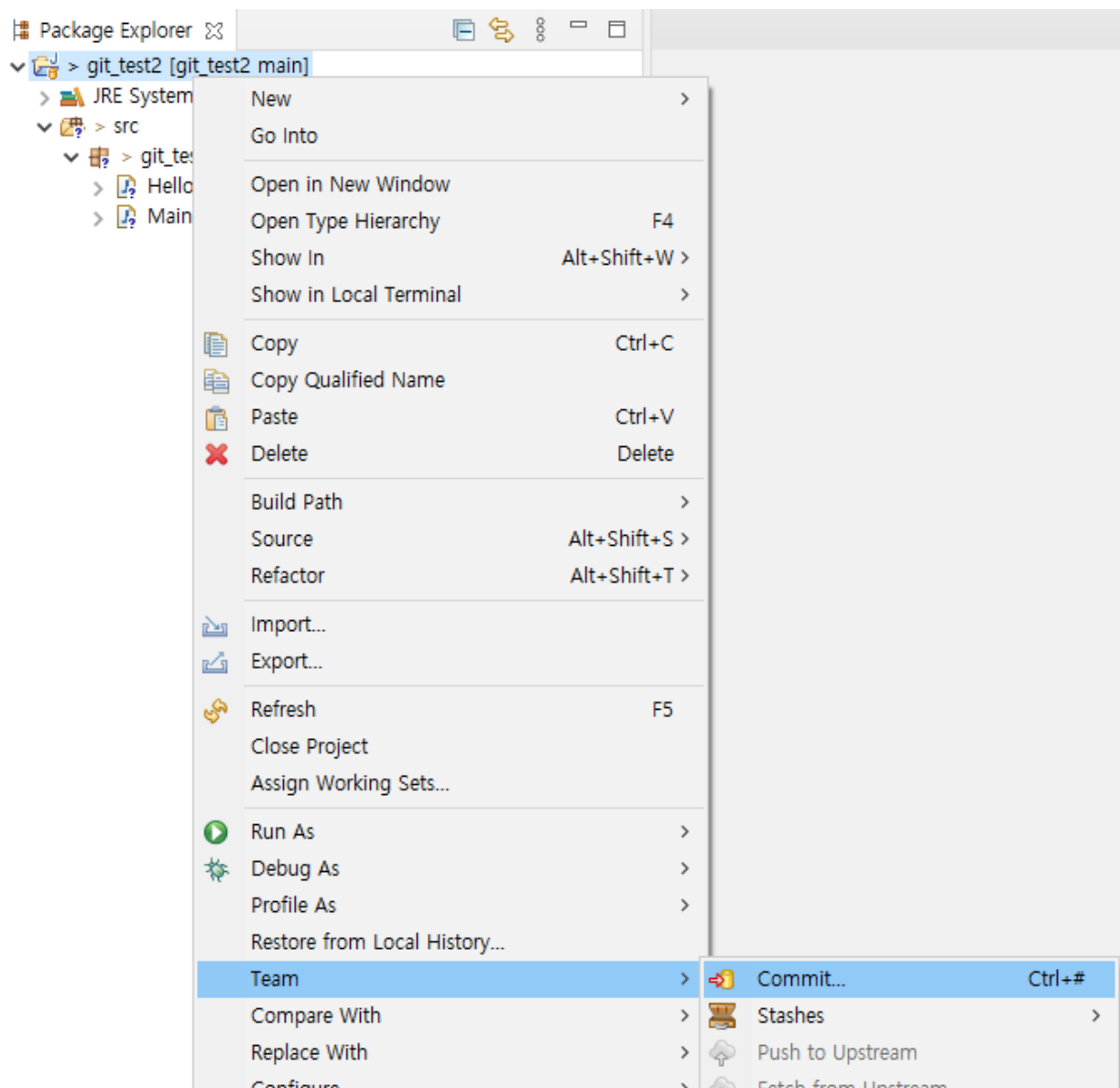
untracked 상태의 파일에 작은 물음표 아이콘이 표시된다.

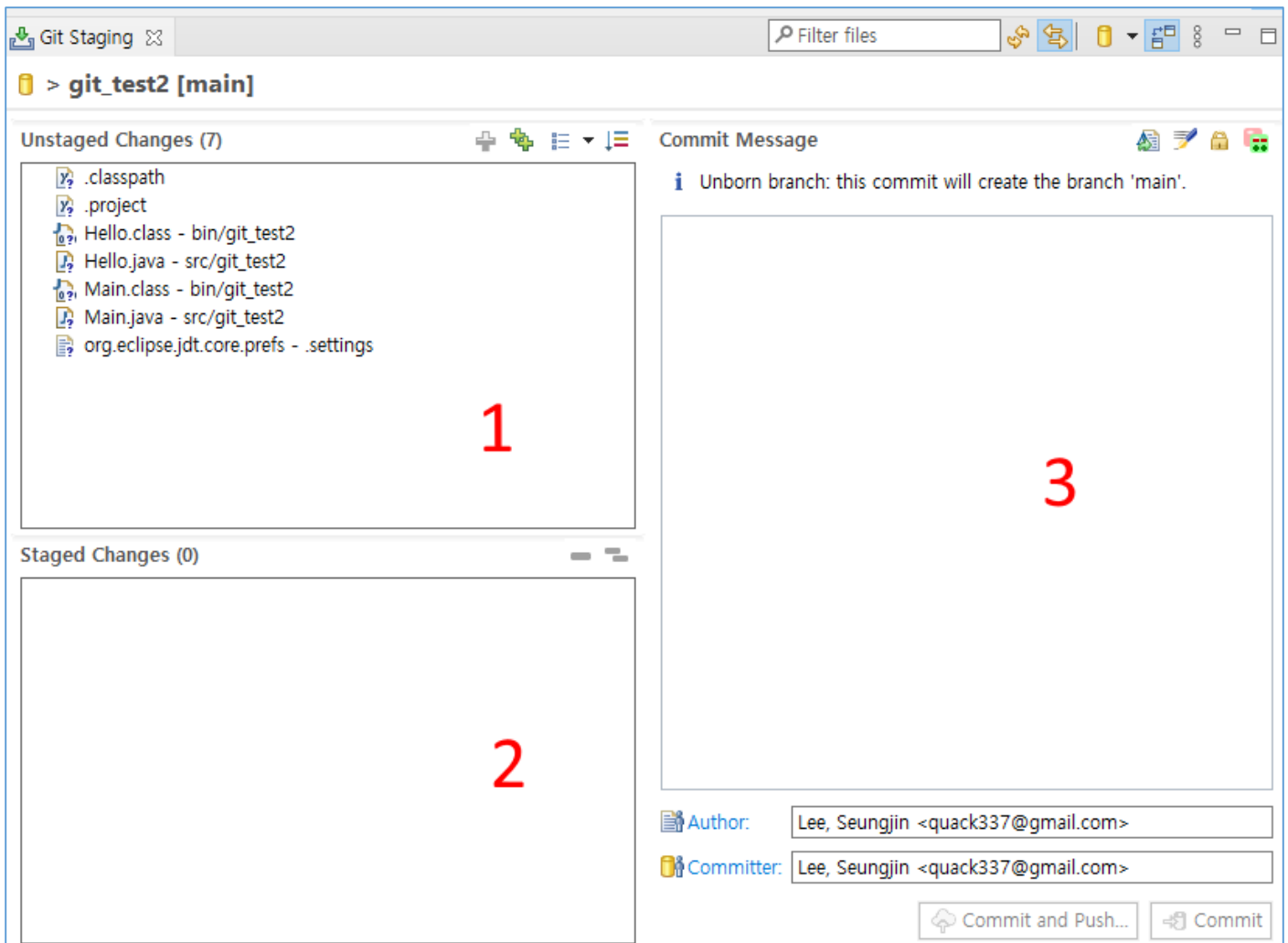
untracked 상태 = 새로 생성되었고 아직 local repository에 기록되지 않은 파일

5) commit

수정된 소스 파일들을 local repository에 등록하는 것을 **commit** 라고 한다.

이클립스 Package Explorer 창에서
프로젝트를 우클릭 - Team - Commit

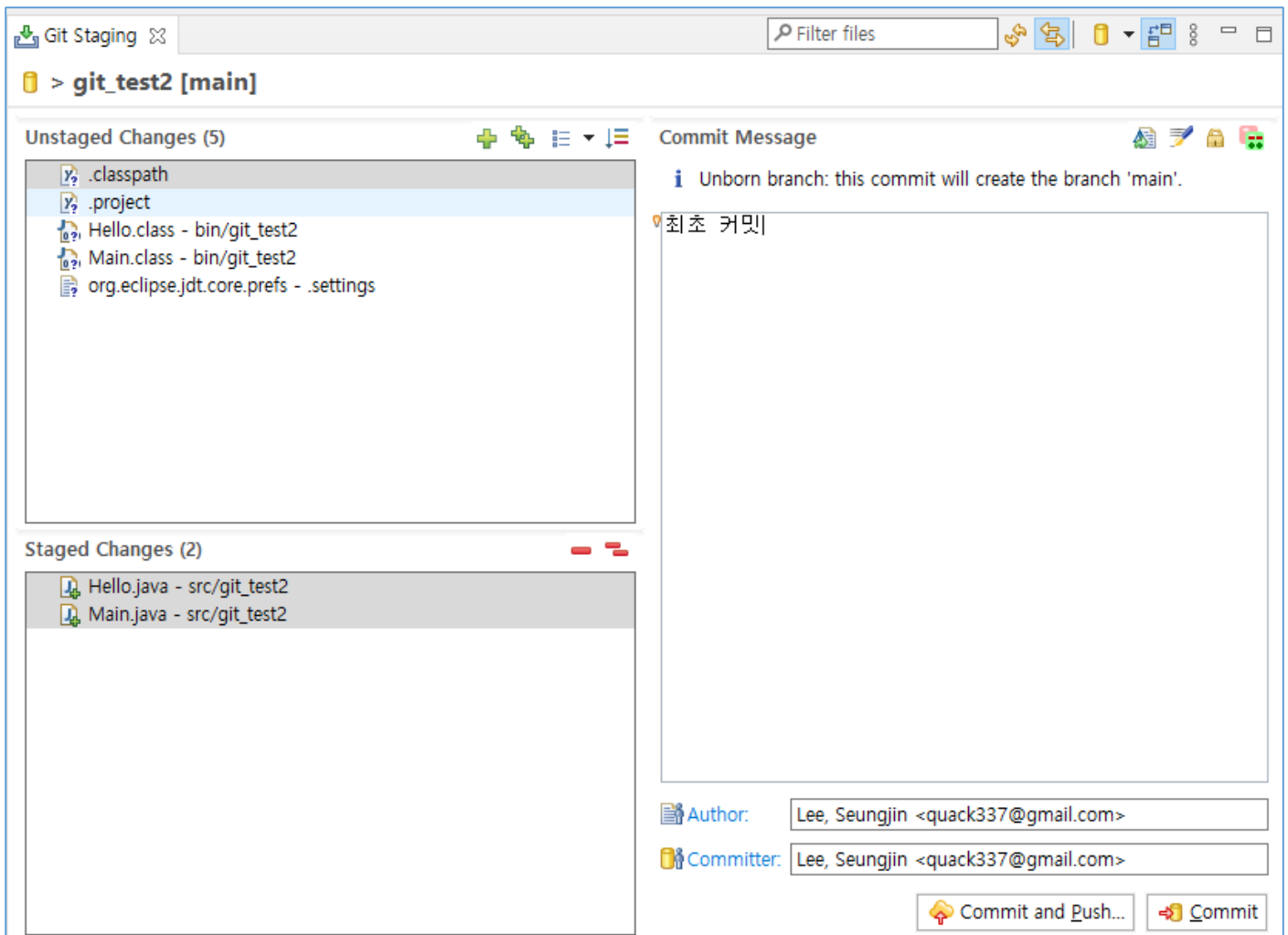




local repository에 등록해야할, 수정된 파일 목록이 영역1에 나열된다.

영역2는 repository에 등록할 파일 목록이다. (staging area)
repository에 등록할 파일들을 영역1에서 영역2로 드래그 드롭해야 한다.

영역3에 commit에 대한 간단한 설명을 적는다.



git local repository에 등록할 파일만 드래그하여 staging area로 이동하자.

소스코드 파일만 git local repository에 등록하자.

Hello.java
Main.java

프로젝트 설정 파일들은 git에 등록하지 않는 것이 좋다.

.classpath
.project
org.eclipse.jdt.core.prefs - .settings

컴파일되어 생성된 바이트코드 파일도 git에 등록하지 않아야 한다.

Hello.class
Main.class

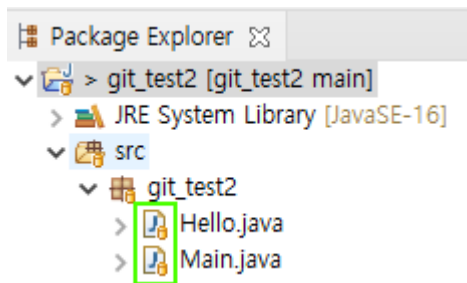
Commit 버튼을 클릭하면,

Staged Changes 영역에 나열된 파일들이 local repository에 등록된다.
수정된 파일들을 local repository에 등록하는 것을 **commit** 라고 한다.

Commit and Push 버튼을 클릭하면,

먼저 local repository에 commit 하고,
바로 이어서 remote repository에 push 한다.

Commit 버튼을 클릭하고, 위 창을 닫자.



파일 아이콘이 unmodified 상태로 바뀌었다.

unmodified 상태 = local repository에 등록되었고, 그 후 수정되지 않았다.

6) .gitignore 파일 생성

git local repository에 등록할 필요가 없는 파일이나 디렉토리 목록을 .gitignore 파일에 입력해 두면 그 파일들이 commit 대상 파일 목록에 아예 등장하지 않고 자동으로 무시되니 편리하다.

프로젝트 설정 파일들은 git에 등록하지 않는 것이 좋다.

.classpath

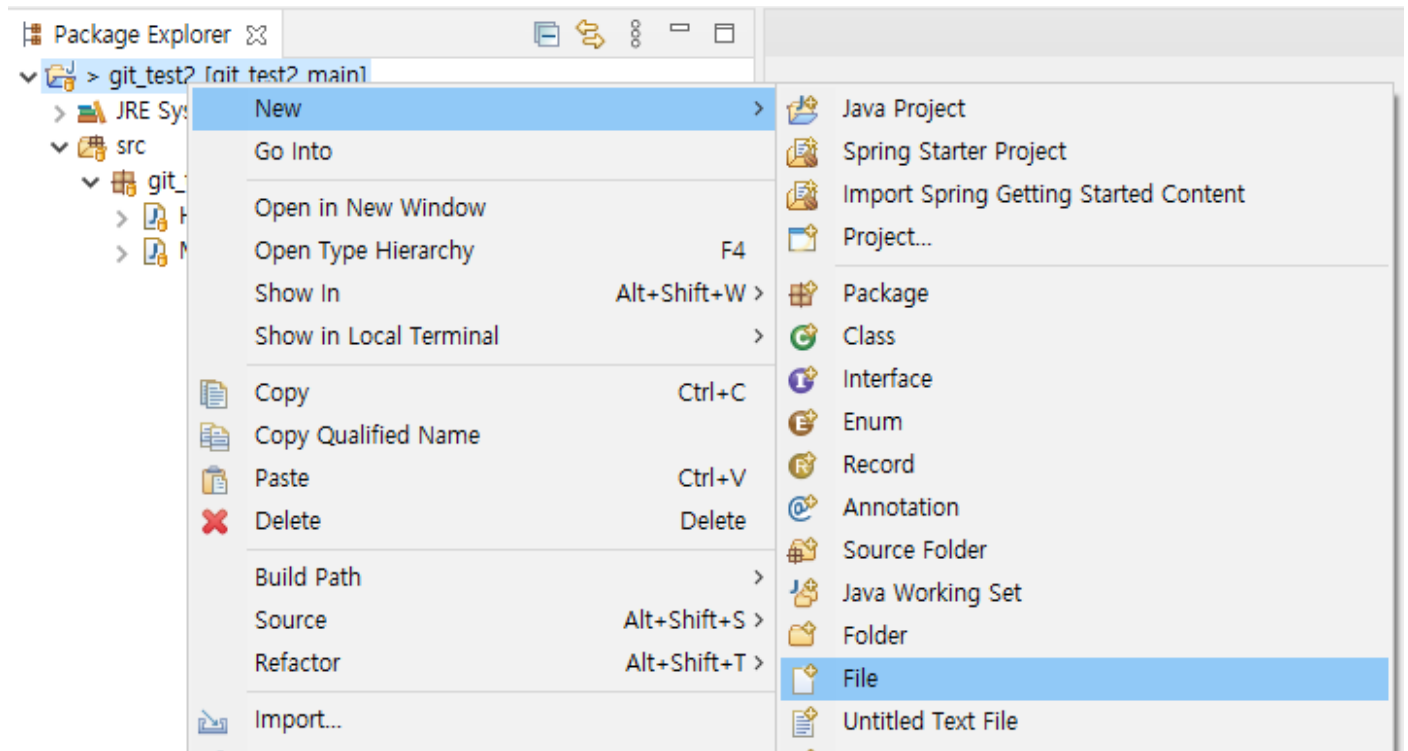
.project

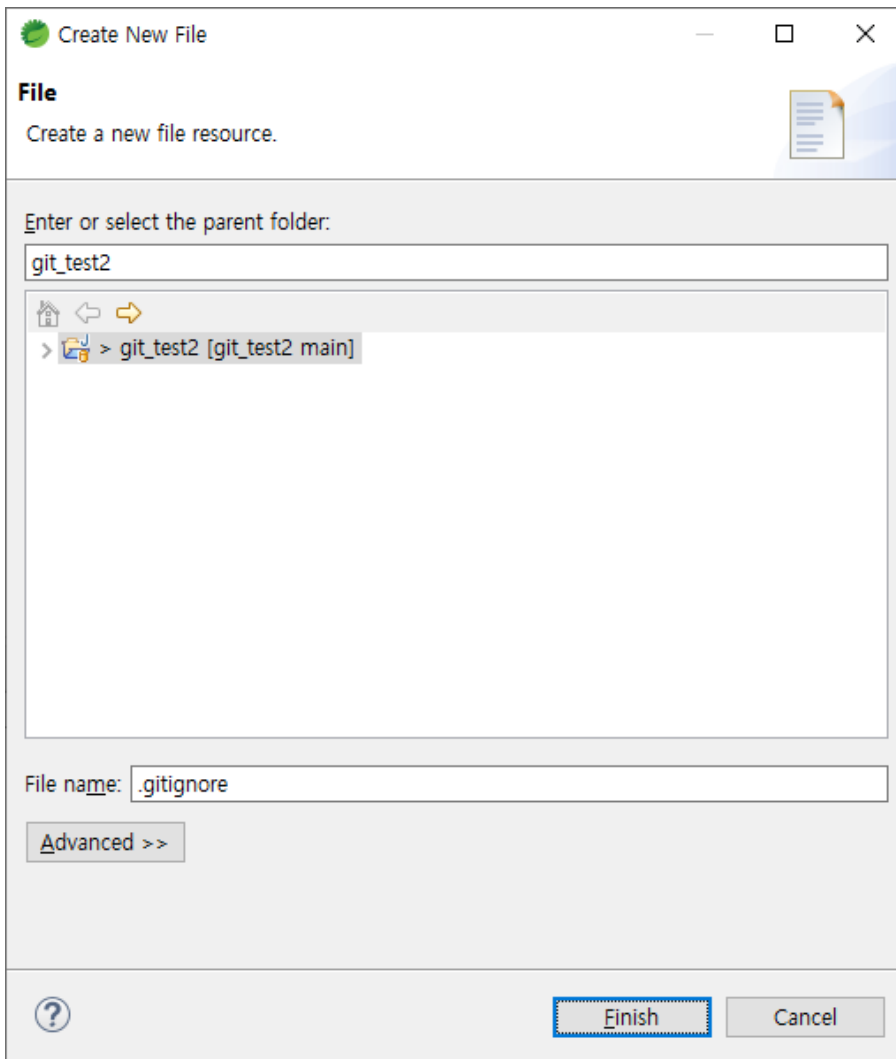
org.eclipse.jdt.core.prefs - .settings (.settings 디렉토리 아래의 파일)

컴파일되어 생성된 바이트코드 파일도 git에 등록하지 않아야 한다.

Hello.class (bin 디렉토리 아래의 파일)

Main.class (bin 디렉토리 아래의 파일)



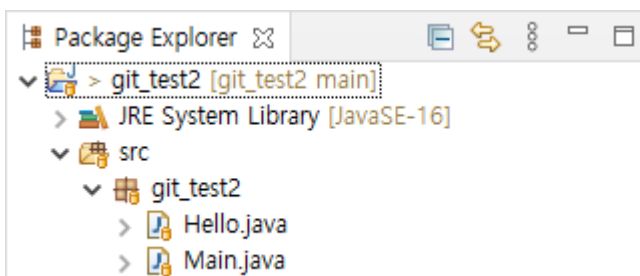


.gitignore

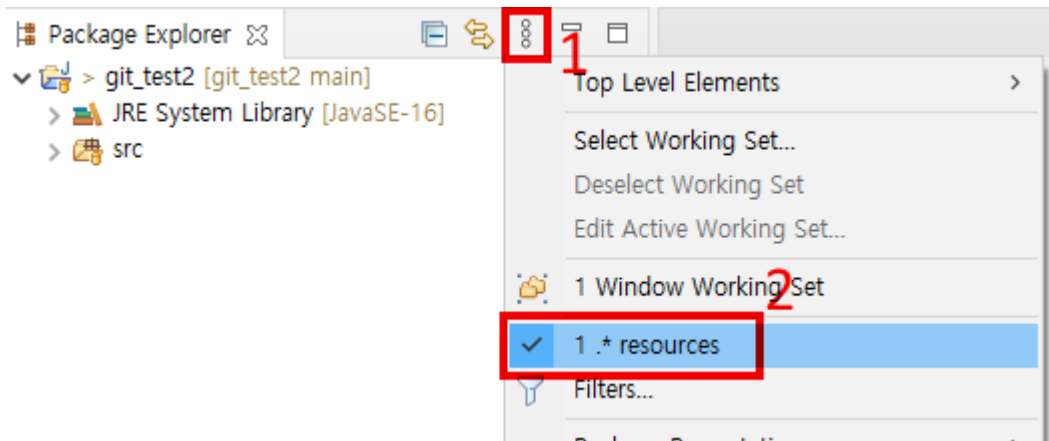
```
bin/
.settings/
.classpath
.project
```

디렉토리 이름 뒤에는 / 문자를 붙여야 한다.

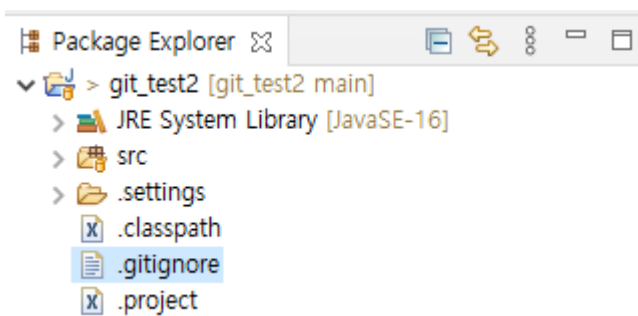
위와 같이 입력하고 Ctrl+S 키를 눌러서 저장하고 창을 닫자.



이름이 점(.)으로 시작하는 파일이나 디렉토리는 위 목록에 표시되지 않는다.

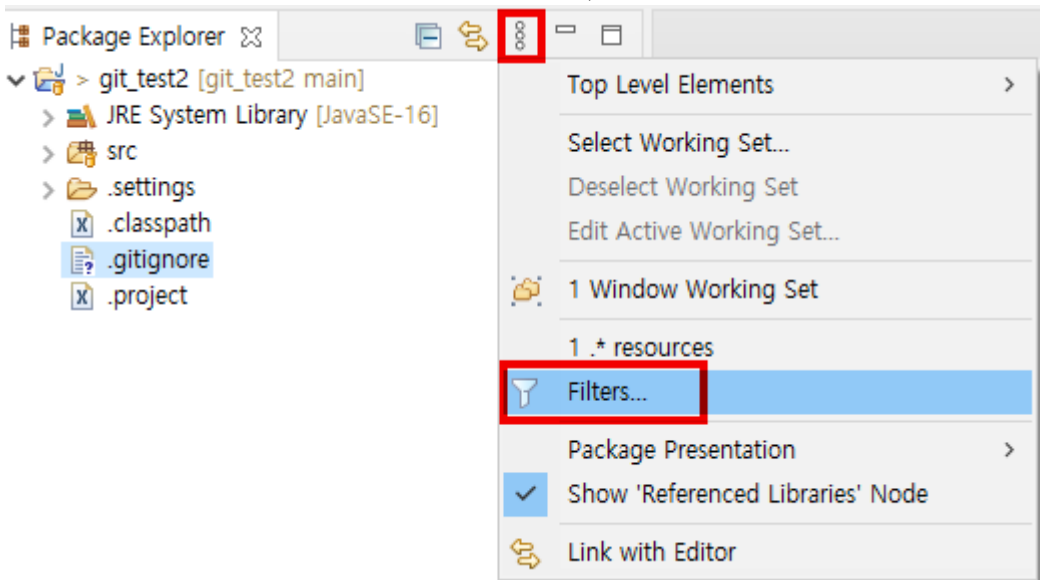


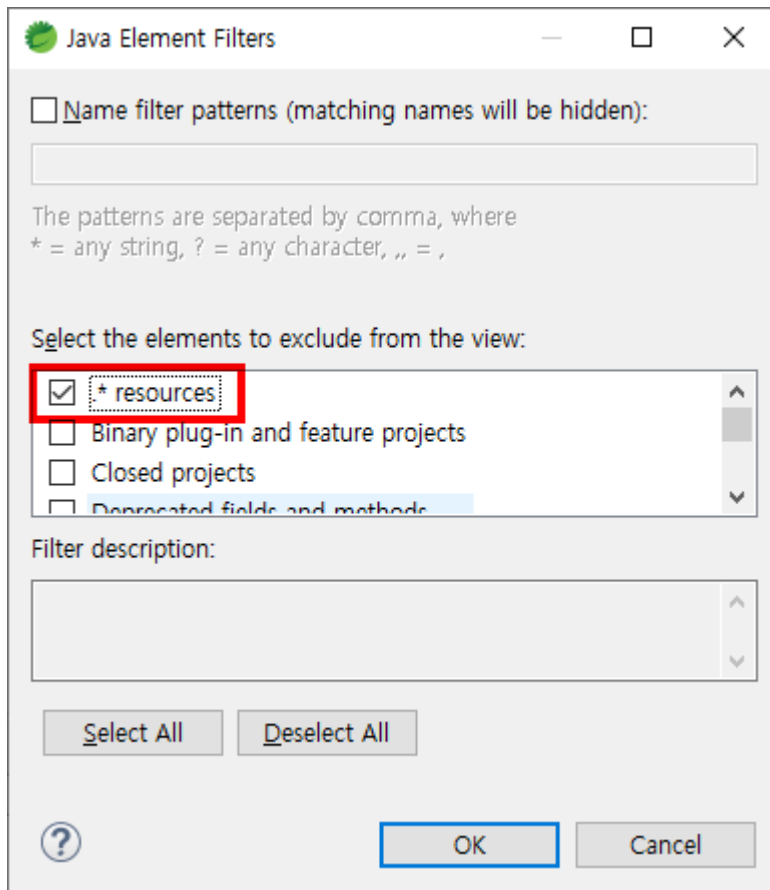
위 화면에서 1 .* resources 메뉴를 클릭하면,
이름이 썸(.)으로 시작하는 파일이나 디렉토리가 위 목록에 표시되고,
이 메뉴를 다시 클릭하면 다시 감춰진다.



.gitignore 파일을 확인할 수 있다.

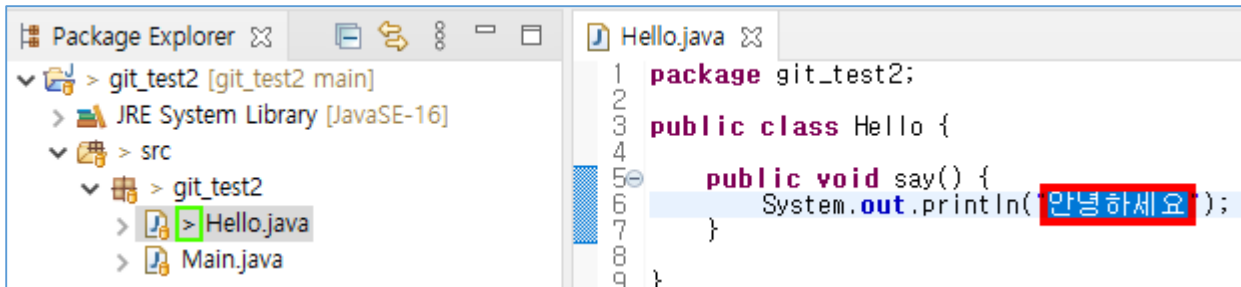
만약 1 .* resources 메뉴가 보이지 않는다면, 대신 Filters 메뉴를 클릭하고





. * resource 항목의 체크를 꺾다 켜도 된다.
체크를 끄면, 썸 파일이 목록에서 감춰지고
체크를 켜면, 썸 파일이 목록에 표시된다.

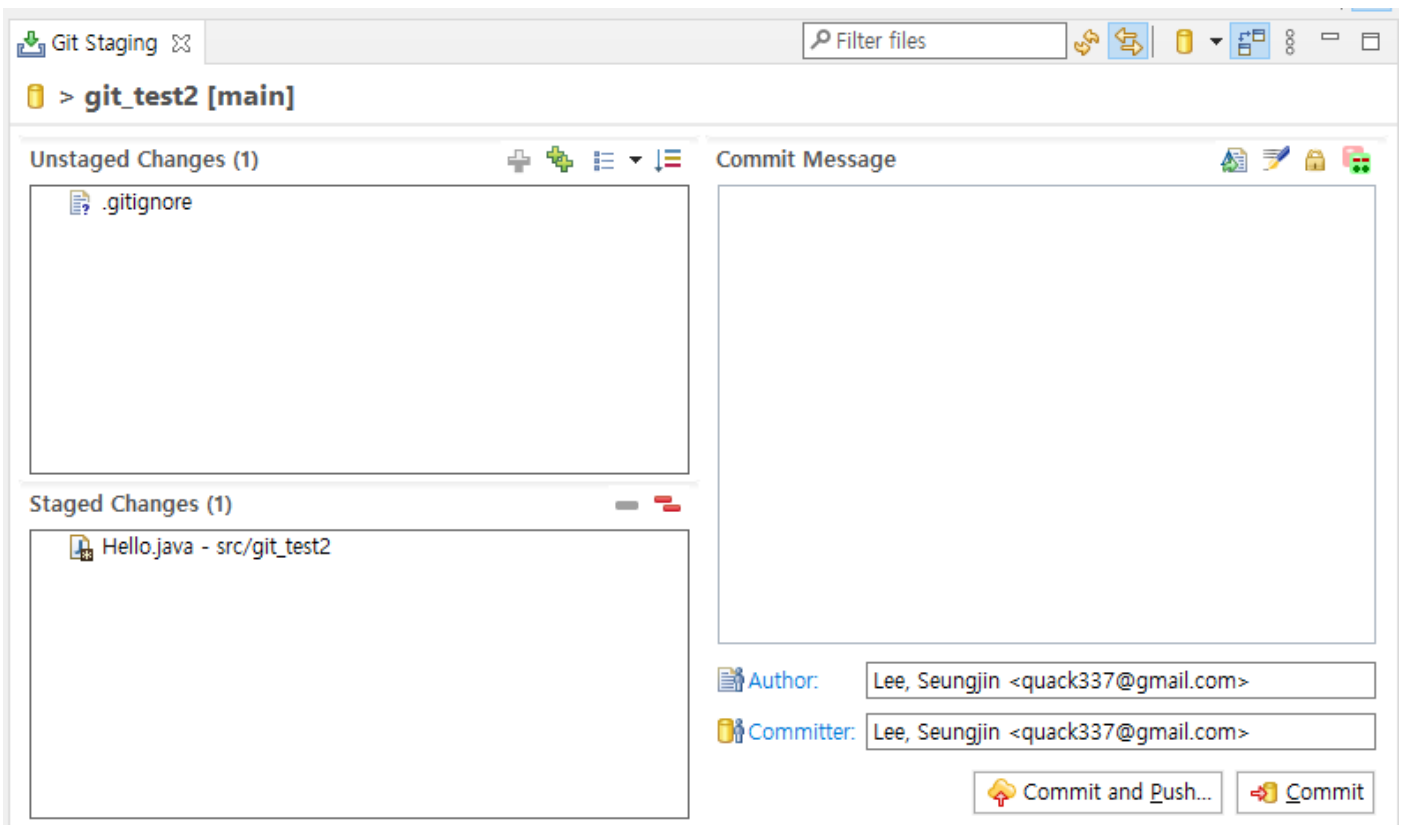
7) Hello.java 수정



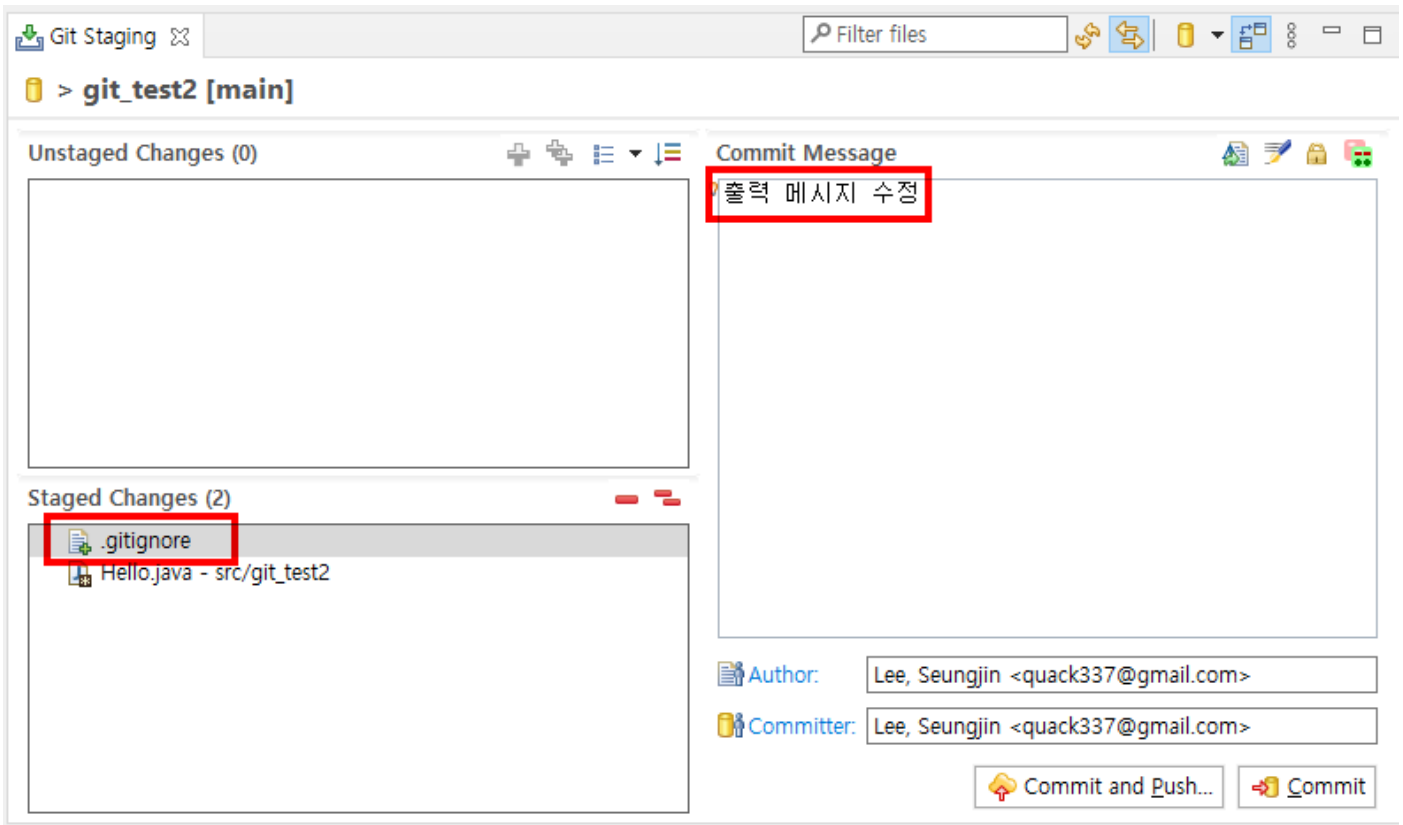
Hello.java 파일을 수정하고 Ctrl+S 키를 눌러서 저장하면,
파일 이름 옆의 아이콘이 modified 상태로 바뀐다.
modified 상태 = git local repository에 기록된 후 수정되었다.

8) 다시 commit

소스코드 수정 작업이 완료되면, 다시 commit 해야 한다.
commit 단축키는 Ctrl+# 이다.

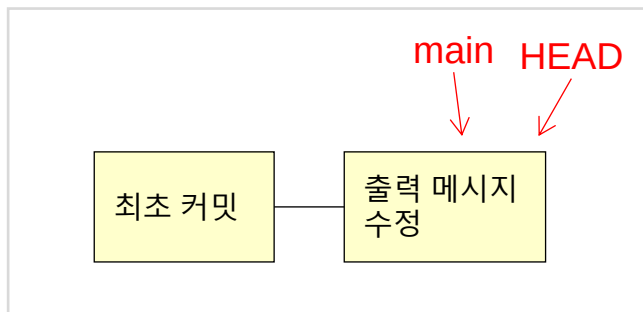


.gitignore 파일에 등록된 프로젝트 설정 파일들, *.class 파일들이
Unstaged Changes 영역에 표시되지 않음에 주목하자.



위와 같이 입력하고,
Commit 버튼을 클릭하고
창을 닫자.

eclipse GUI local repository



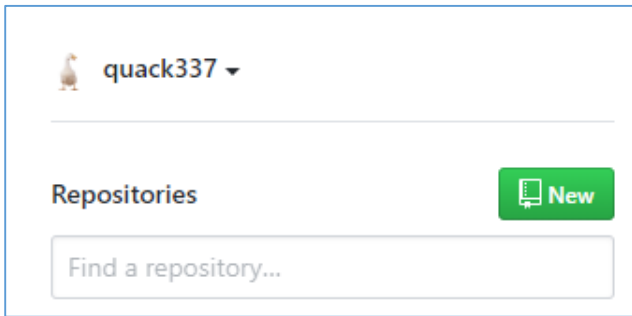
remote repository



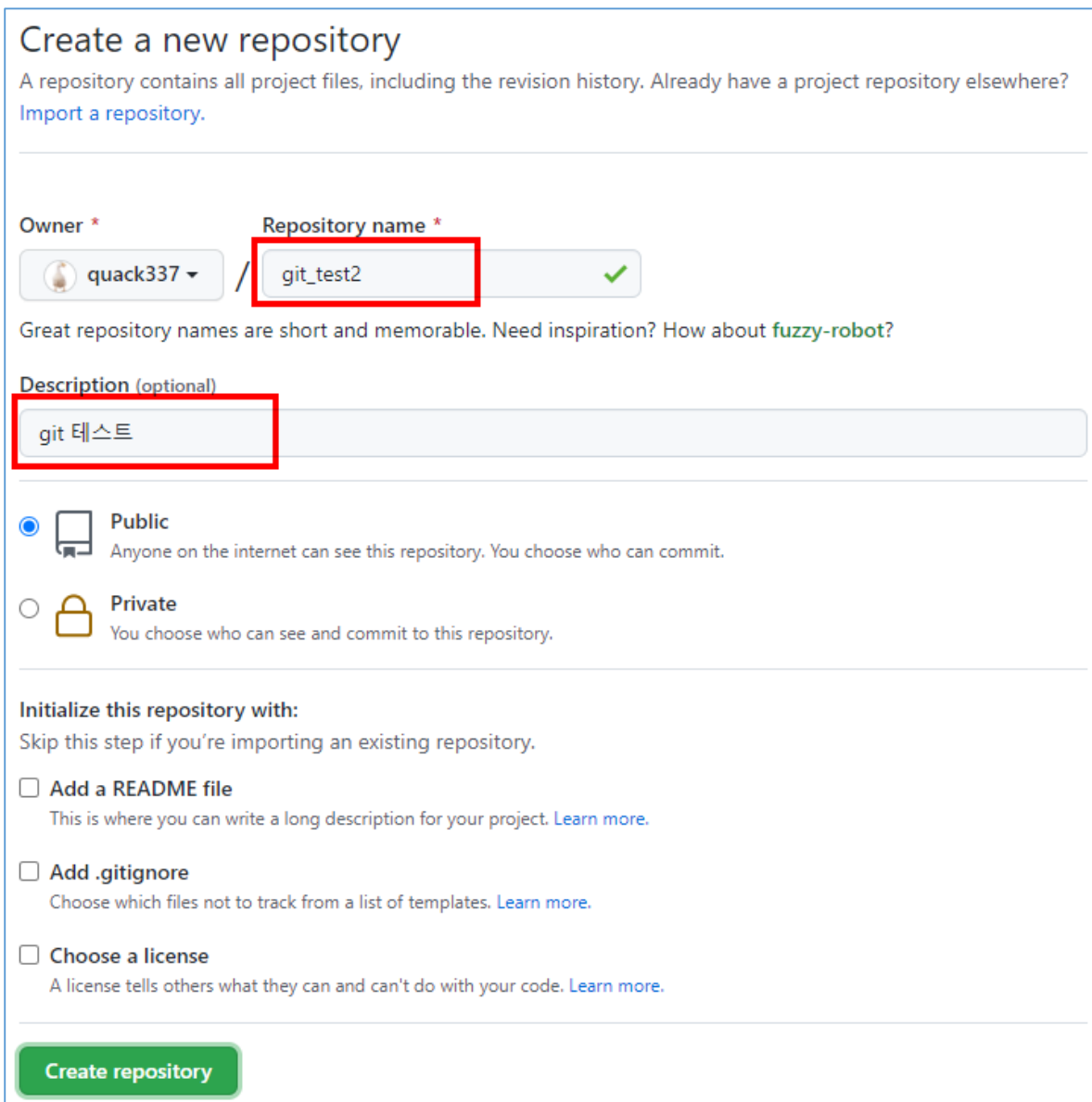
main : main branch 의 최신 commit
HEAD : 현재 branch 의 최신 commit
노란색 : local repository에만 등록된 commit

3. remote repository 생성

1) github에 repository 생성



New 클릭



Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Owner * Repository name *

quack337 / git_test2 ✓

Great repository names are short and memorable. Need inspiration? How about [fuzzy-robot?](#)

Description (optional)

git 테스트

☒ Public
Anyone on the internet can see this repository. You choose who can commit.

☐ Private
You choose who can see and commit to this repository.

Initialize this repository with:

Skip this step if you're importing an existing repository.

☐ Add a README file
This is where you can write a long description for your project. [Learn more.](#)

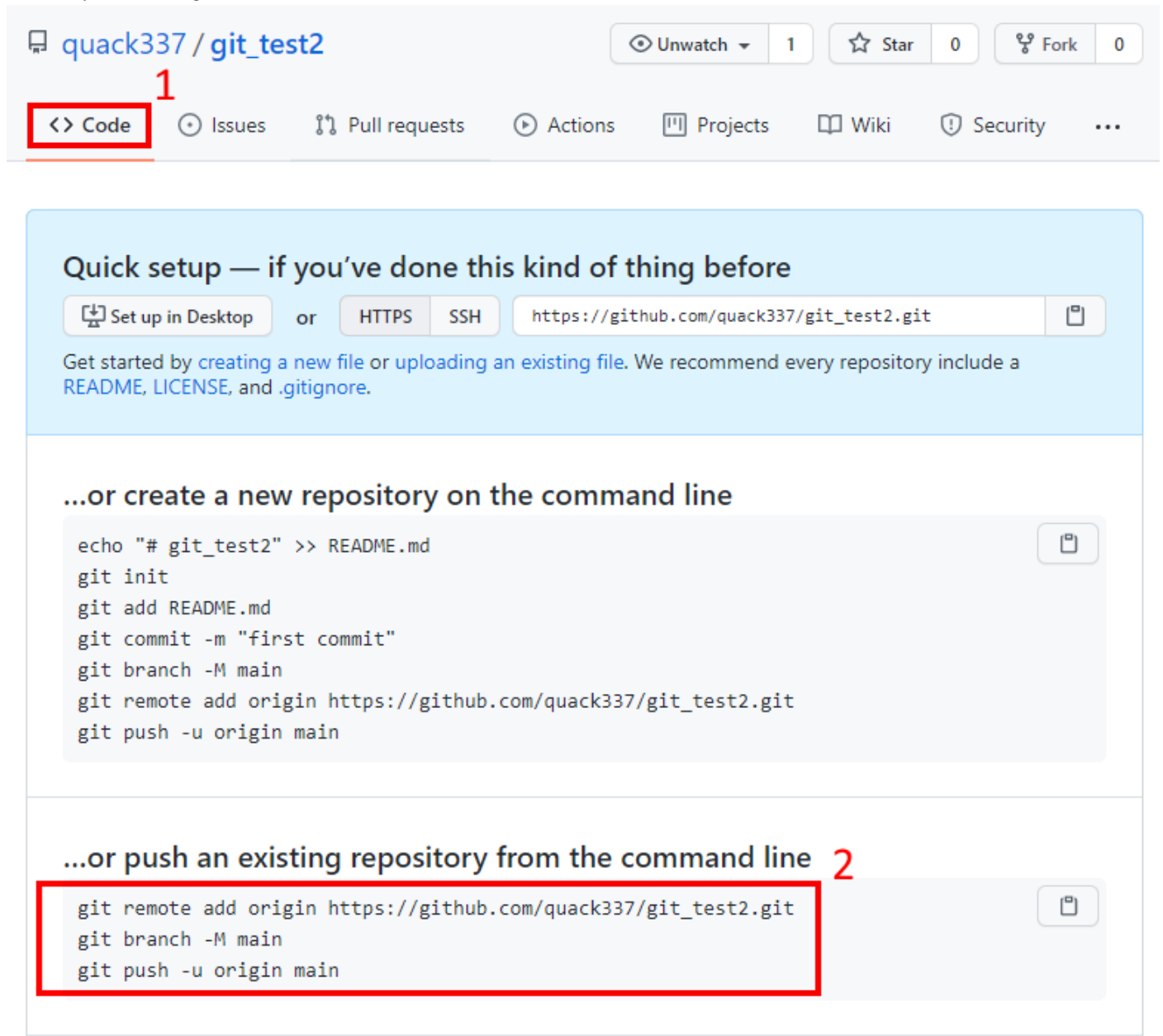
☐ Add .gitignore
Choose which files not to track from a list of templates. [Learn more.](#)

☐ Choose a license
A license tells others what they can and can't do with your code. [Learn more.](#)

Create repository

Create repository 버튼을 클릭하면, Github 서버에 리포지토리가 생성된다.
이것을 remote repository라고 부른다.

2) repository URL 확인



quack337 / git_test2

Unwatch 1 Star 0 Fork 0

<> Code Issues Pull requests Actions Projects Wiki Security ...

Quick setup — if you've done this kind of thing before

Set up in Desktop or HTTPS SSH https://github.com/quack337/git_test2.git

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a README, LICENSE, and .gitignore.

...or create a new repository on the command line

```
echo "# git_test2" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/quack337/git_test2.git
git push -u origin main
```

...or push an existing repository from the command line

```
git remote add origin https://github.com/quack337/git_test2.git
git branch -M main
git push -u origin main
```

아직 업로드된 코드가 없다면, Code 탭을 눌렀을 때 위 화면이 나타난다.

local repository를 remote repository에 연결하고
최초 push 하는 명령을 확인할 수 있다.

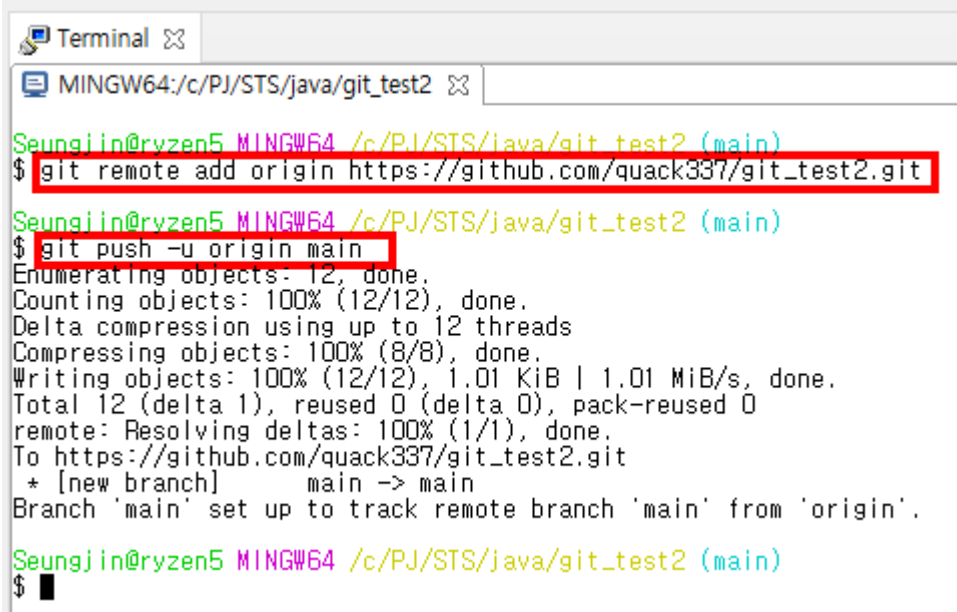
```
git remote add origin https://github.com/quack337/git_test2.git
git push -u origin main
```

우리가 만든 local repository의 기본 브랜치 이름은 이미 main 이기 때문에
git branch -M main 명령은 필요 없다. (이 명령은 현재 브랜치 이름을 main 으로 변경하는 것)

4. remote repository에 push

1) 최초 push

local repository에 commit 한 내용을 remote repository에 push 하자.

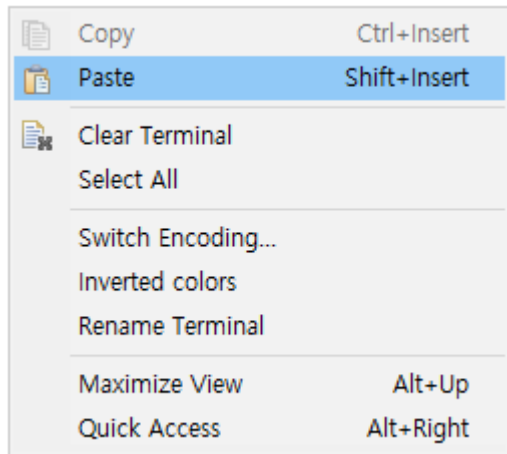


```
Terminal
MINGW64:/c/PJ/STS/java/git_test2
Seungjin@ryzen5 MINGW64 /c/PJ/STS/java/git_test2 (main)
$ git remote add origin https://github.com/quack337/git_test2.git
Seungjin@ryzen5 MINGW64 /c/PJ/STS/java/git_test2 (main)
$ git push -u origin main
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 12 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (12/12), 1.01 KiB | 1.01 MiB/s, done.
Total 12 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/quack337/git_test2.git
 * [new branch]      main -> main
Branch 'main' set up to track remote branch 'main' from 'origin'.
Seungjin@ryzen5 MINGW64 /c/PJ/STS/java/git_test2 (main)
$
```

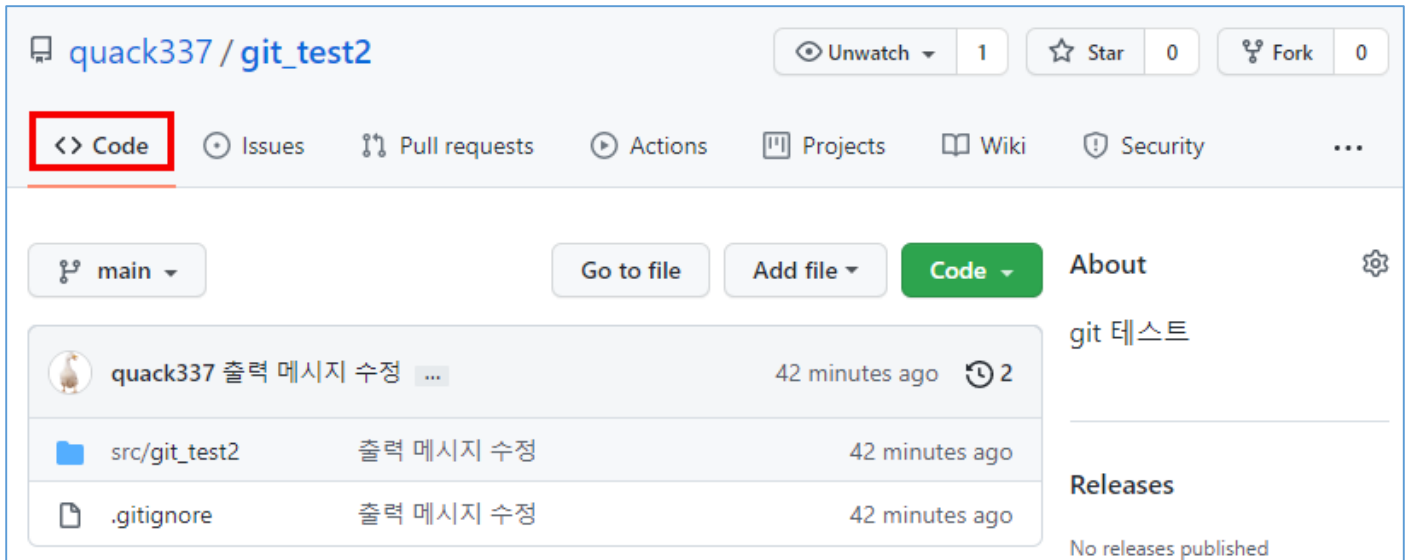
git bash 쉘에서 아래 명령을 순서대로 실행하자.

```
git remote add origin https://github.com/quack337/git_test2.git
git push -u origin main
```

위 터미널 창을 오른쪽 버튼으로 클릭하면 paste 메뉴가 보인다.

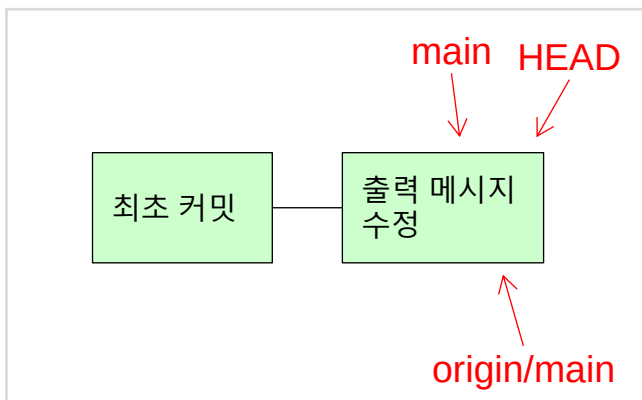


2) push 결과 확인

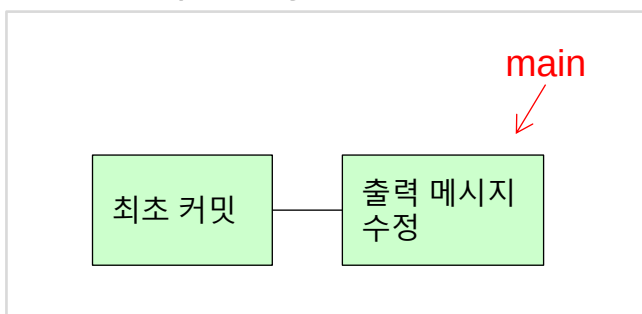


업로드된 코드가 있다면, Code 탭에서 위와 같은 화면을 볼 수 있다.

eclipse GUI local repository



remote repository



녹색 : remote repository 와 동기화된 commit
main : main branch 의 최신 commit
HEAD : 현재 branch 의 최신 commit
origin/main : 마지막 동기화 시점에서 remote repository 의 main 브랜치의 최신 commit

동기화 = remote repository 로부터 download OR remote repository 에 upload

5. 프로젝트 가져오기

아래 절차는 팀장이 remote repository에 등록한 프로젝트 소스 코드를 팀원 PC로 가져오는 절차이다.
혼자 실습해야 하니, git_test2 프로젝트를 삭제하고 다시 다운로드 하자.

github에서 프로젝트 가져오기 #1, #2 방법은 별로 추천할만 하지 않다.

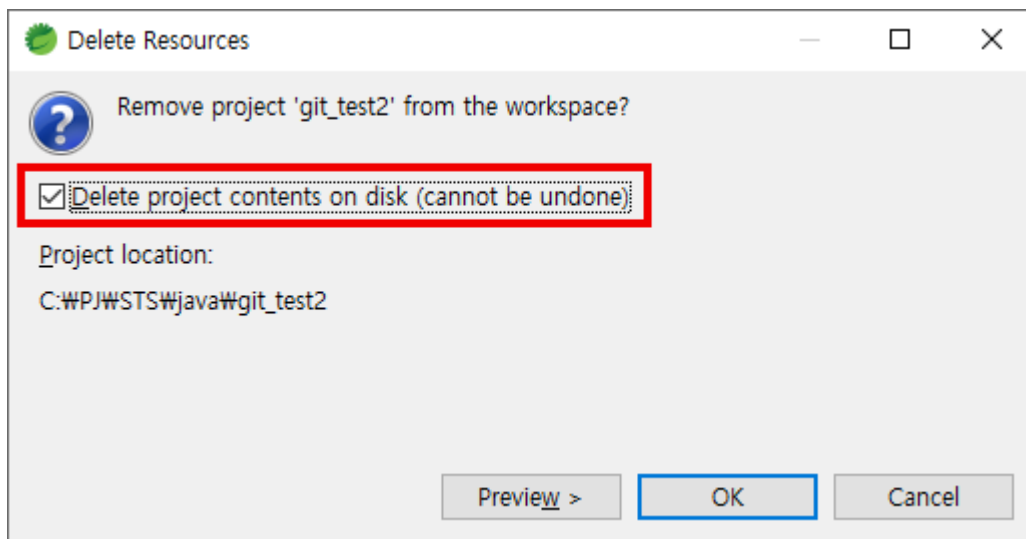
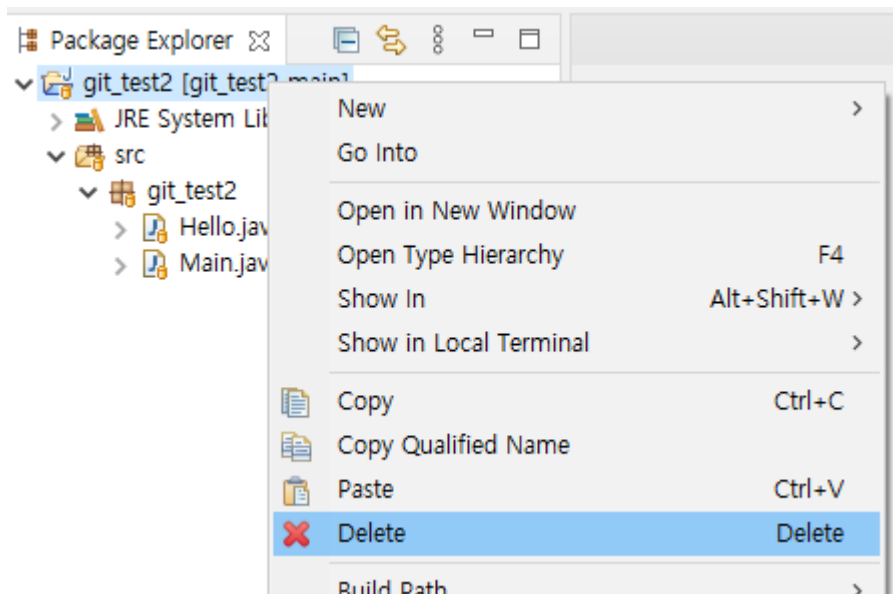
백엔드 spring boot 프로젝트 같은 고급 프로젝트의 경우 잘 작동하지 않는 경우가 많다.

github에서 프로젝트 가져오기 #3, #4 방법을 추천한다.

1) git_test2 프로젝트 삭제

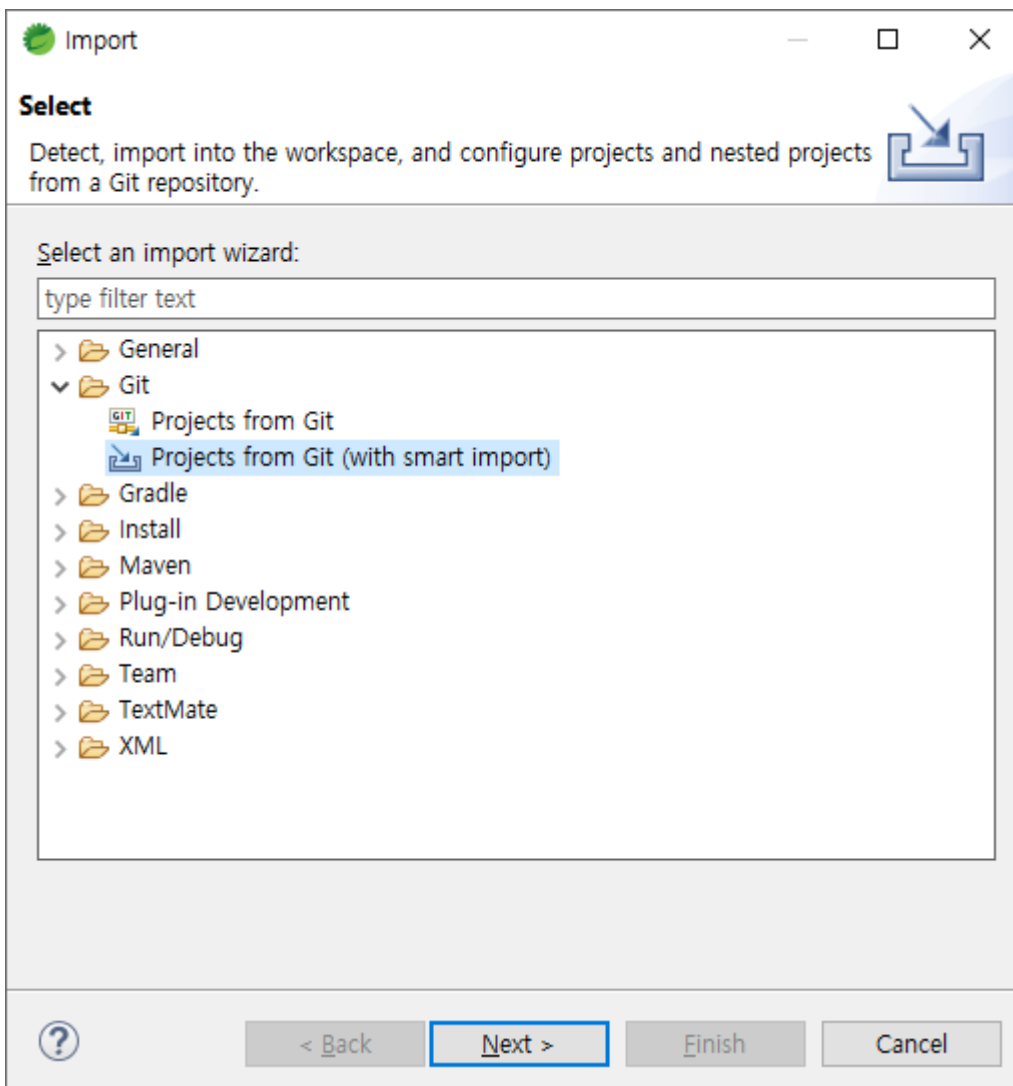
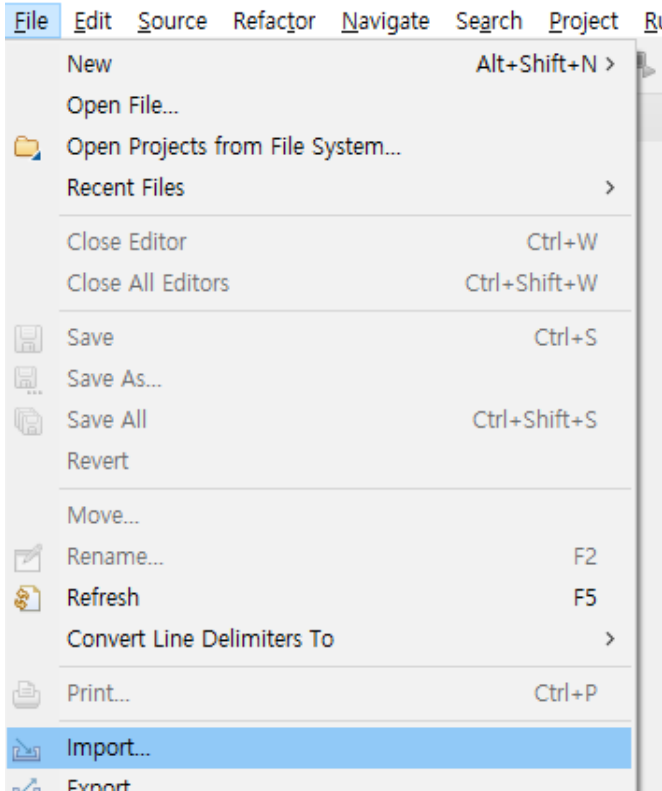
열려 있는 소스코드 창을 전부 닫자.

터미널 창도 닫자.



Ok 클릭

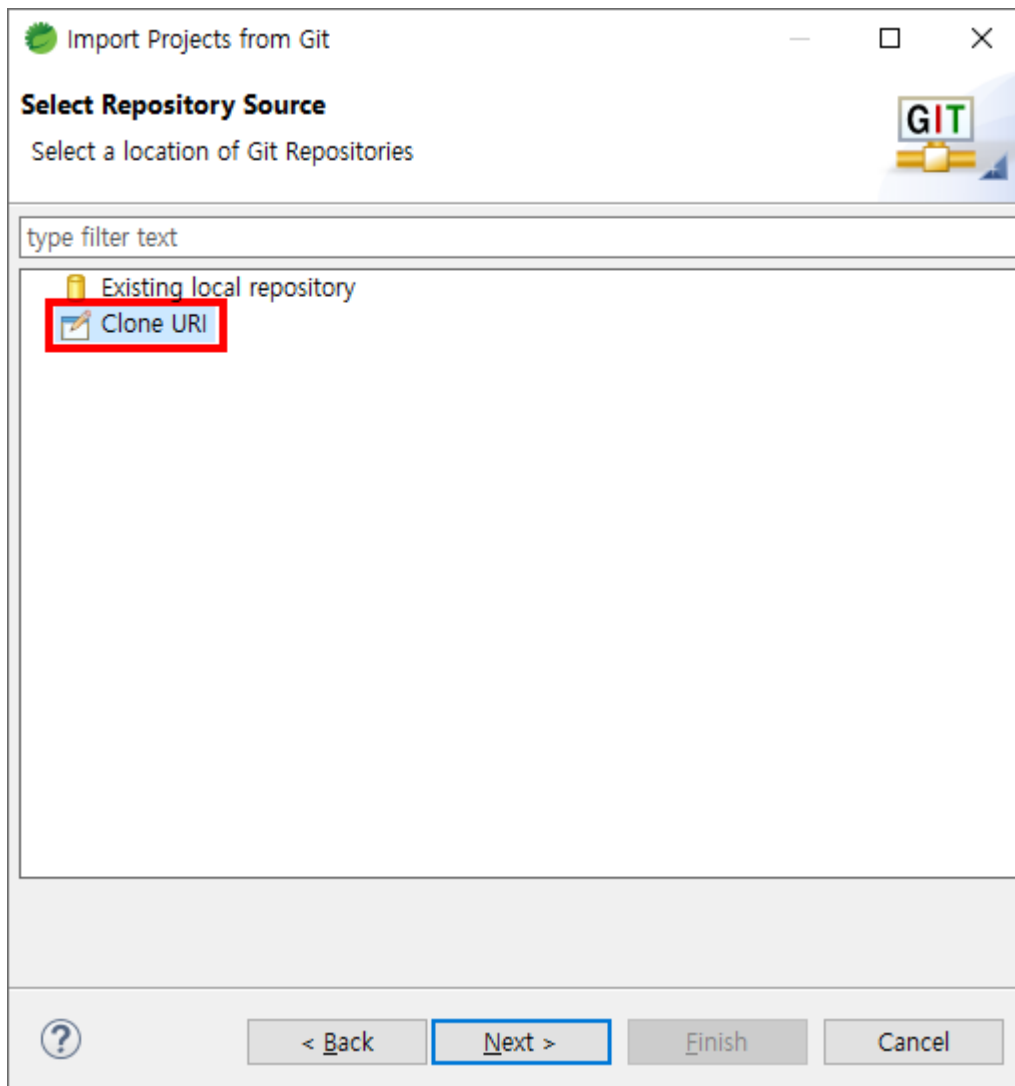
2) github에서 프로젝트 가져오기 #1



Next 클릭

github에 push된 git_test2 프로젝트에서, 프로젝트 설정 파일들이 들어있지 않다.
github에서 프로젝트를 가져올 때, 프로젝트 설정 파일을 새로 만들어야 한다.

Projects from Git (with smart import) 항목을 선택하면,
eclipse가 프로젝트 유형을 짐작하여 프로젝트 설정 파일들을 자동으로 만들어 준다.



Clone URI를 선택하고
Next 클릭

Import Projects from Git

Source Git Repository

Enter the location of the source repository.

Location

URI: Local Folder... Local Bundle File...

Host:

Repository path:

Connection

Protocol: ▼

Port:

Authentication

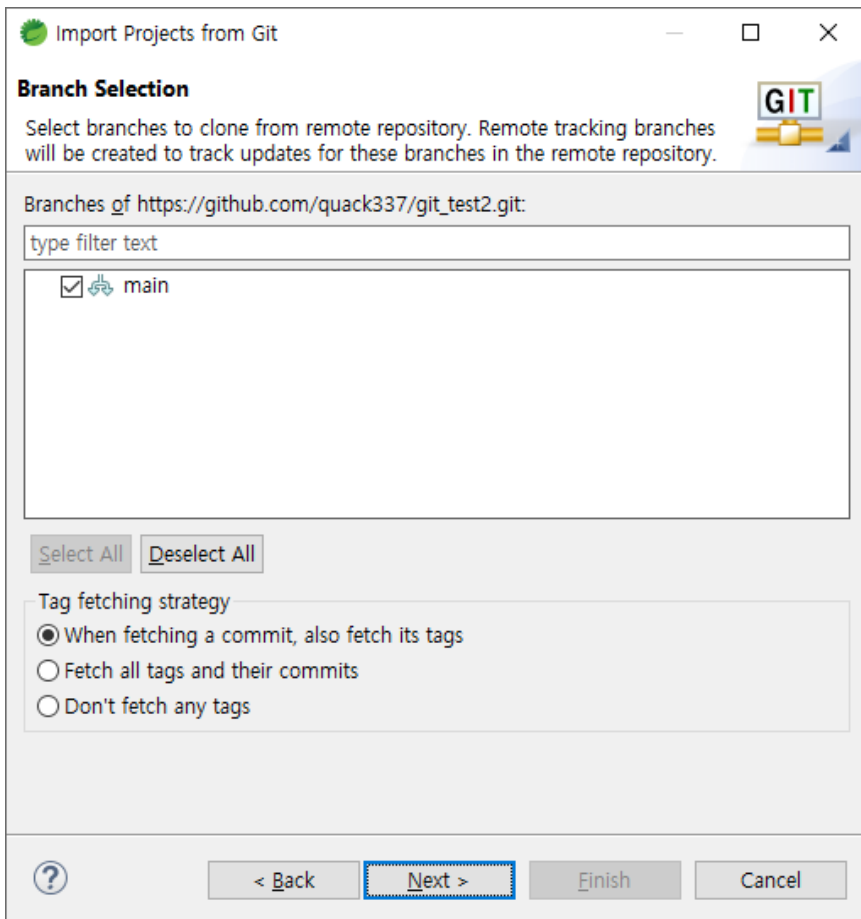
User:

Password:

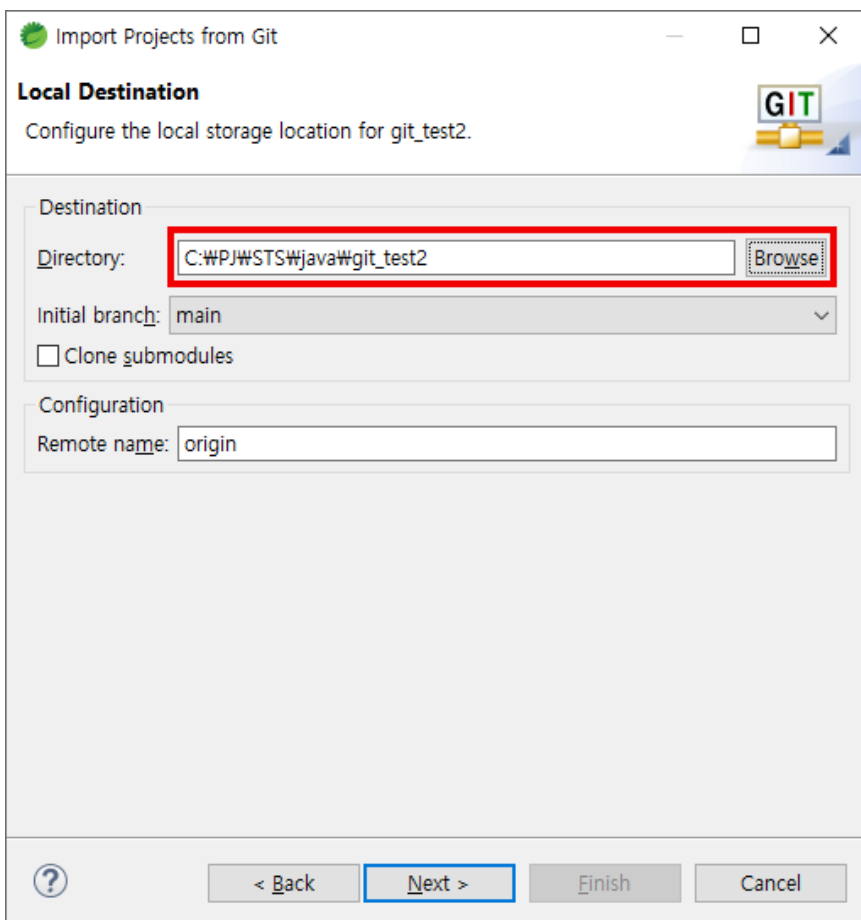
☒ Store in Secure Store

github repository의 URL을 입력한다. (예: `https://github.com/quack337/git_test2.git`)
자신의 github 계정을 입력한다.
Store in Secure Store 항목을 체크한다.

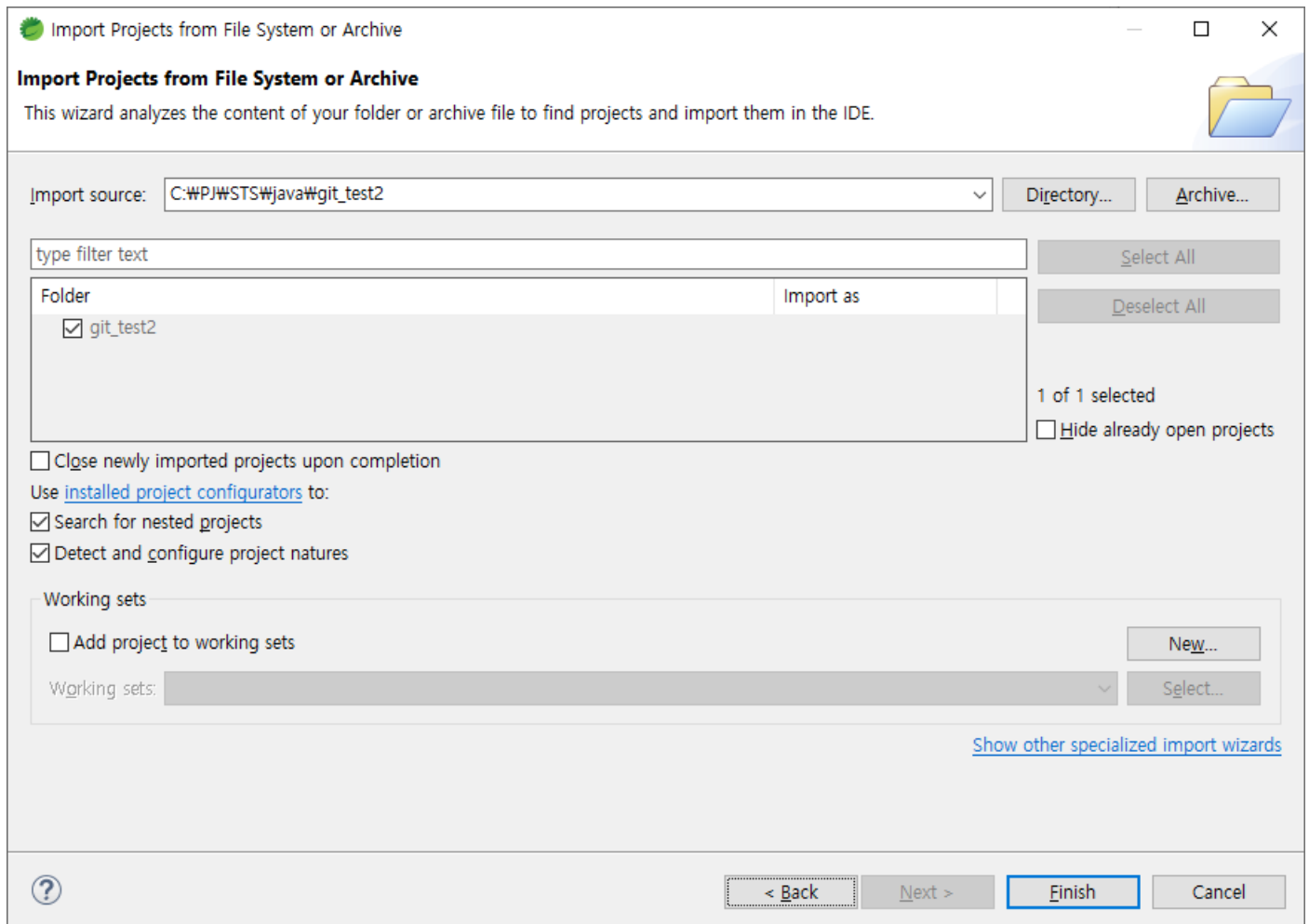
Next 클릭



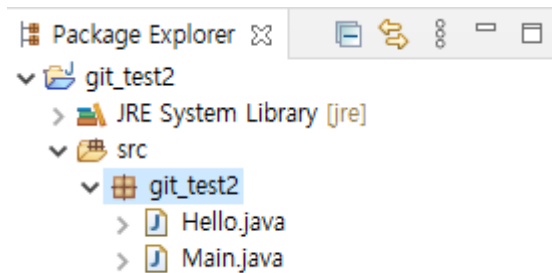
Next 클릭



프로젝트를 저장할 적당한 폴더를 선택하고
Next 클릭



Finish 클릭



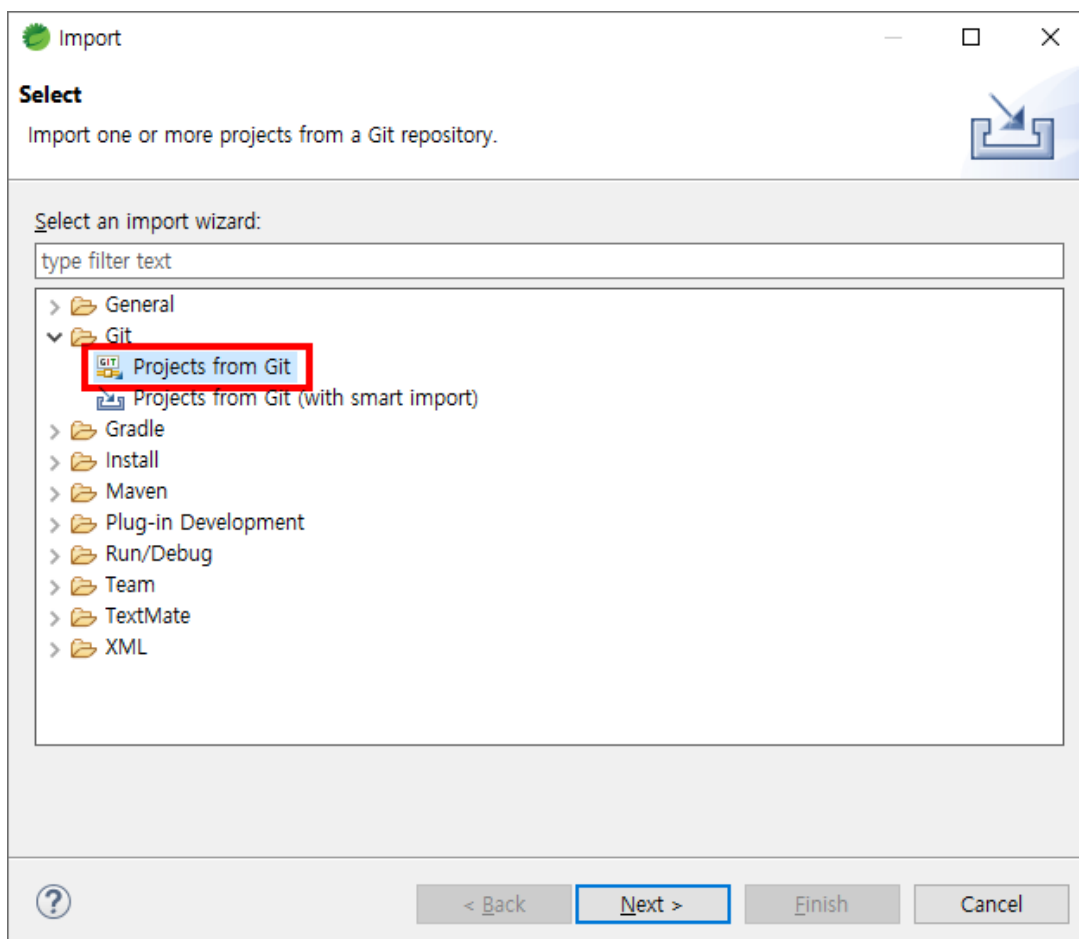
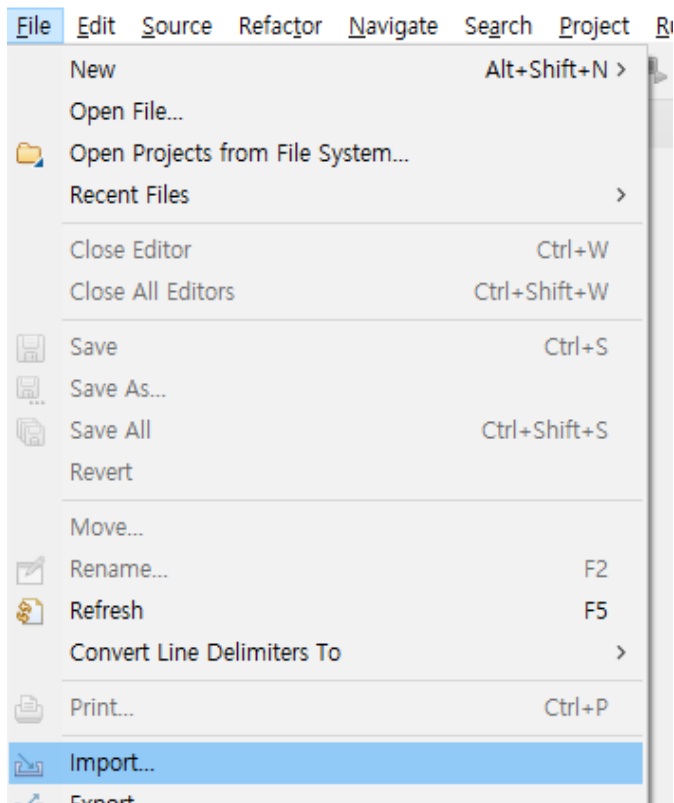
완료

만약 자동으로 생성된 프로젝트 설정 파일에 문제가 있다면,
프로젝트를 완전히 삭제한 후, 아래에서 설명하는 방법으로 프로젝트를 가져와야 한다.

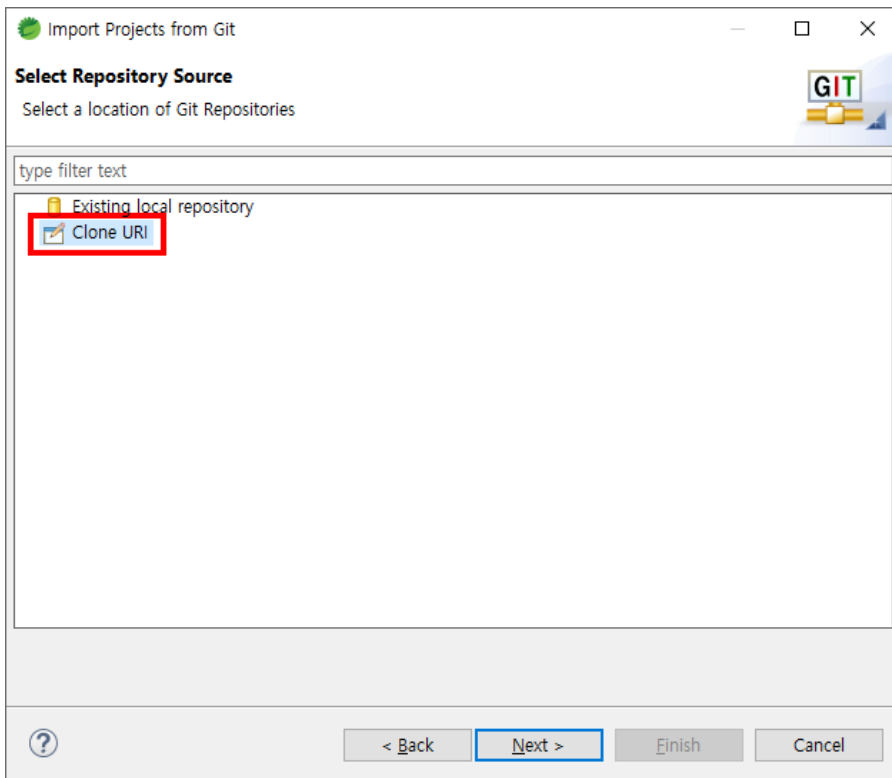
3) github에서 프로젝트 가져오기 #2

위 #1 절차가 실패한 경우에는 #2 절차로 프로젝트를 가져와야 한다.

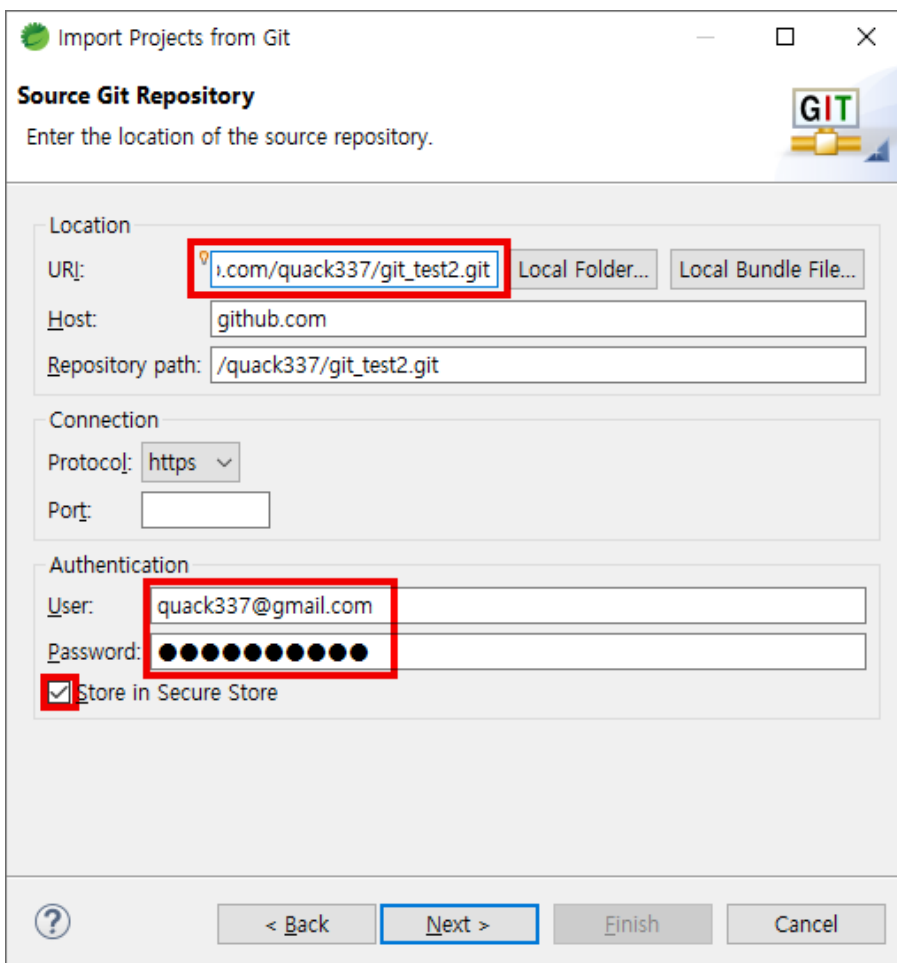
위 #1 절차가 성공했다면 #2 절차를 또 실행할 필요 없다.



Next 클릭

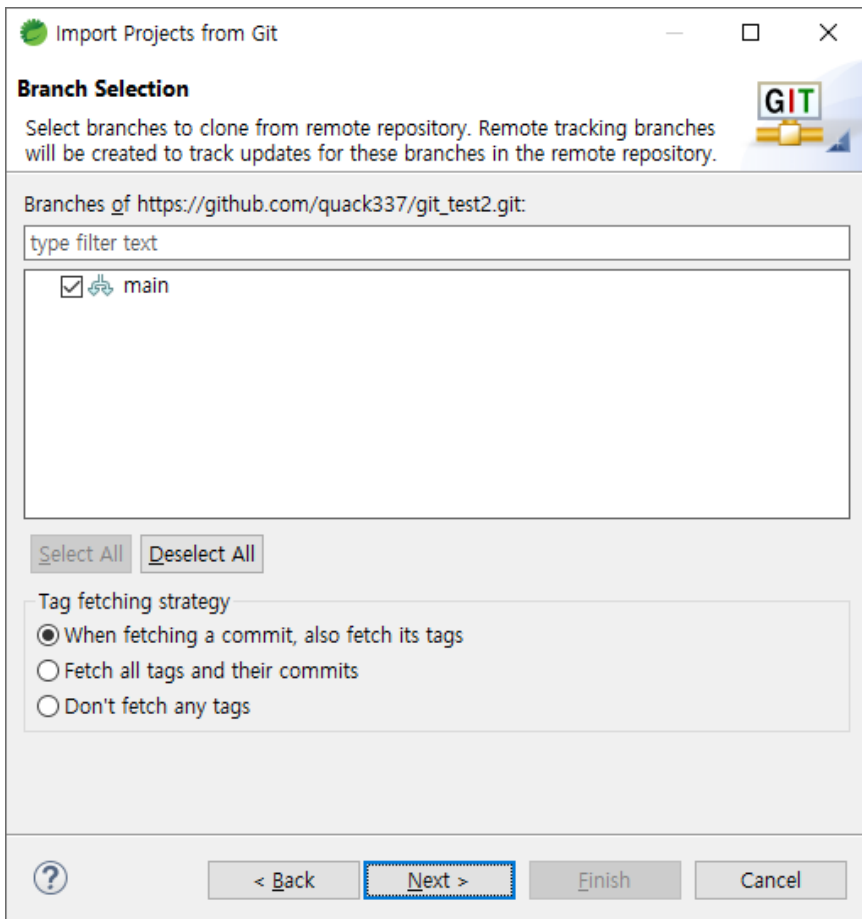


Next 클릭

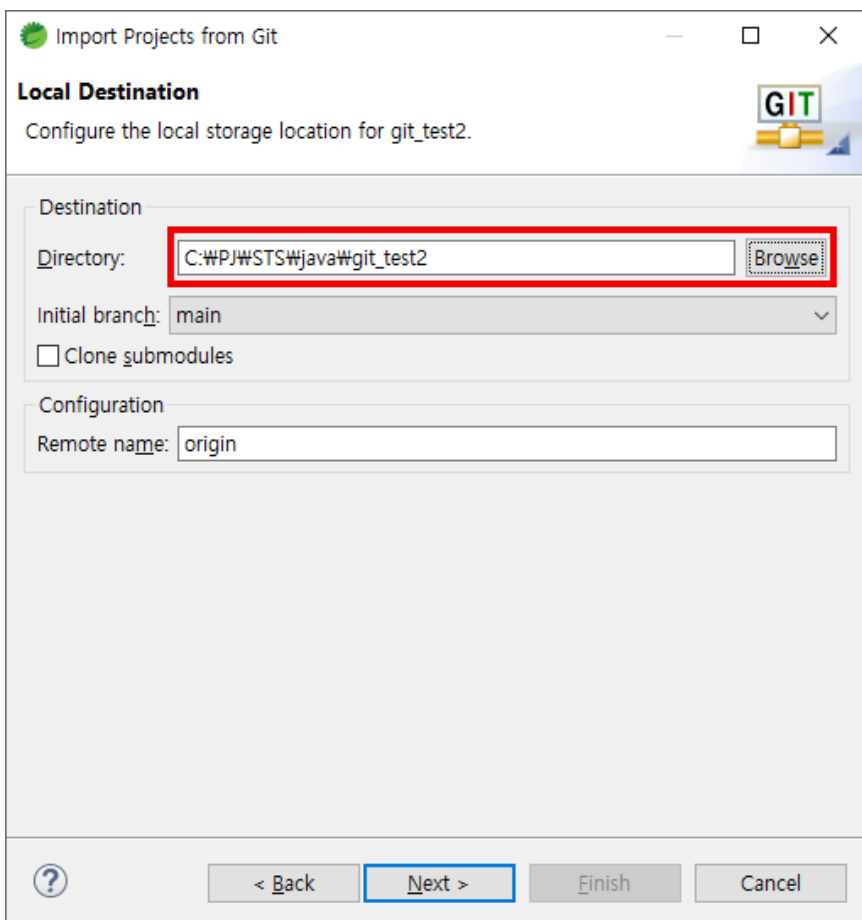


github repository의 URL을 입력한다. (예: `https://github.com/quack337/git_test2.git`)
자신의 github 계정을 입력한다.
Store in Secure Store 항목을 체크한다.

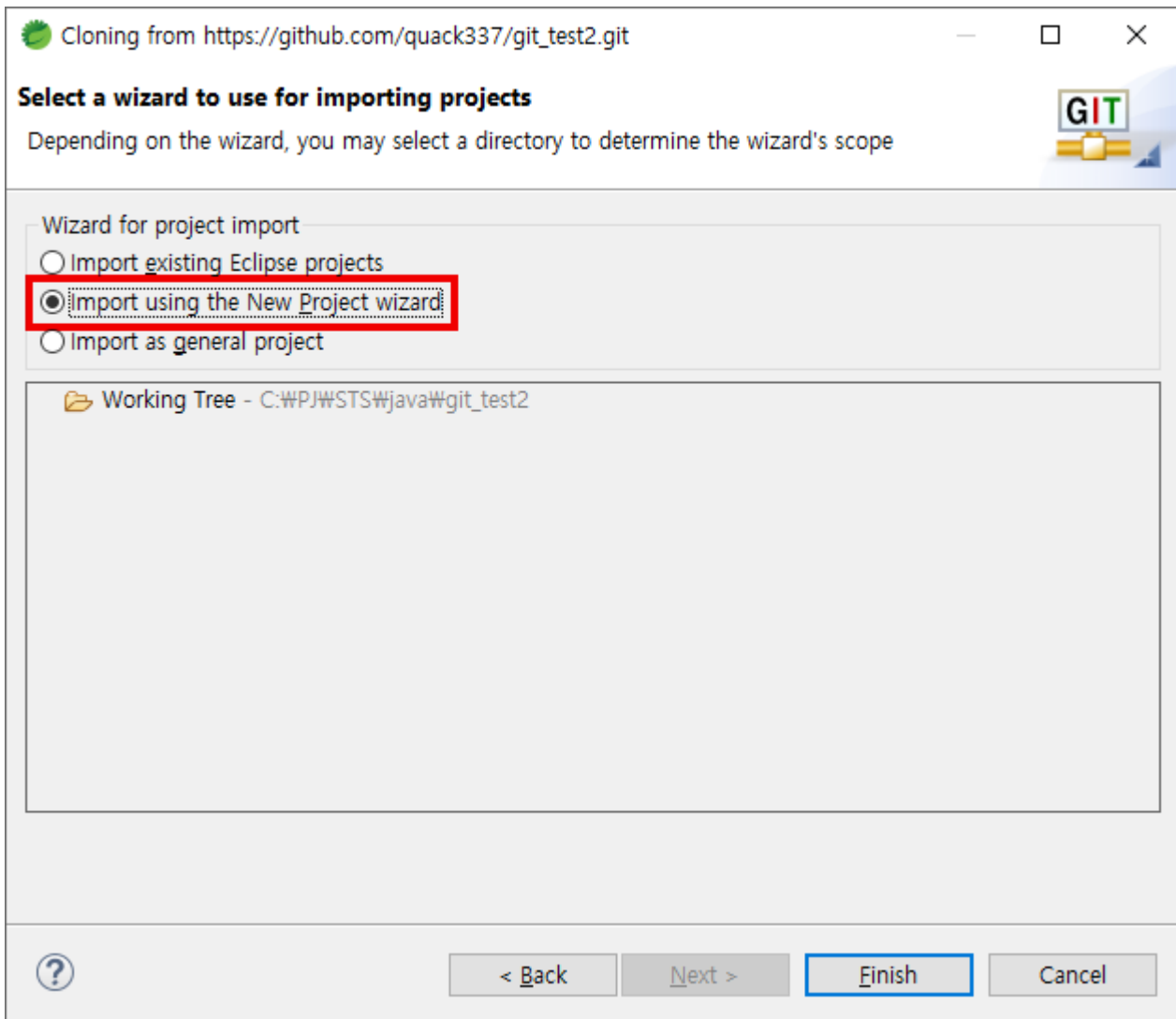
Next 클릭



Next 클릭



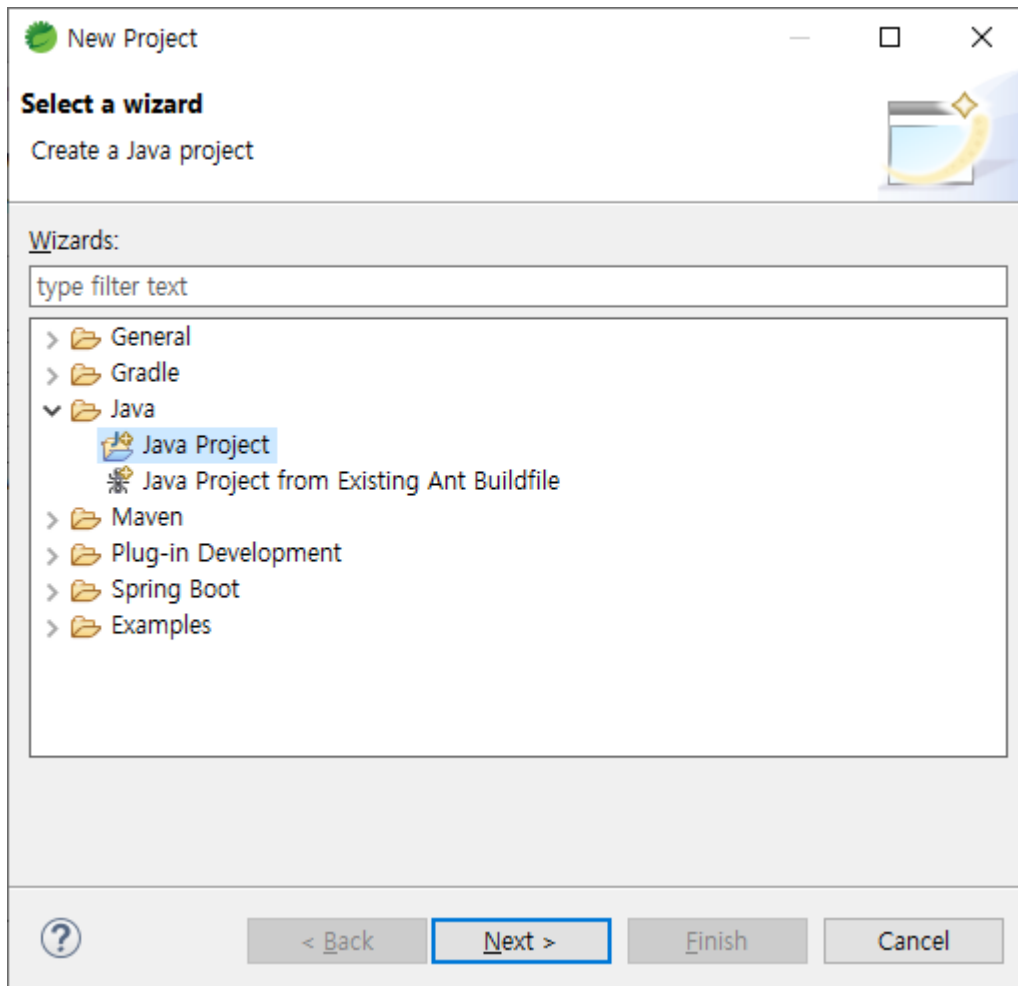
프로젝트를 저장할 적당한 폴더를 선택하고
Next 클릭



Import using the New Project wizard 항목을 선택하고
Finish 클릭

참고

java 소스코드 파일만 git에 등록했고, 프로젝트 설정 파일은 제외했다.
그래서 remote repository에서 프로젝트를 다운로드 받을 때, 프로젝트 설정 파일을 새로 만들어야 한다.
import using the new project wizard 항목이 바로 프로젝트 설정 파일을 새로 만들기 위한 선택이다.



이 화면에서 생성할 프로젝트의 유형을 선택해야 한다.
Java Project 항목을 선택하고
Next 클릭

New Java Project

Create a Java Project

Create a Java project in the workspace or in an external location.

Project name:

☒ Use default location

Location:

JRE

☒ Use an execution environment JRE:

☐ Use a project specific JRE:

☐ Use default JRE 'jre' and workspace compiler preferences [Configure JREs...](#)

Project layout

☐ Use project folder as root for sources and class files

☒ Create separate folders for sources and class files [Configure default...](#)

Working sets

☐ Add project to working sets

Working sets:

Module

☒ Create module-info.java file

i The wizard will automatically configure the JRE and the project layout based on the existing source.

새로 생성할 프로젝트의 이름을 입력하고 (git_test2)
Finish 클릭

New module-info.java

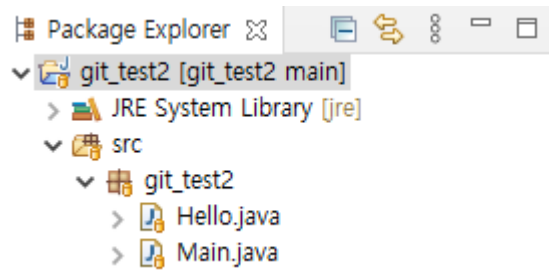
Create module-info.java

Create a new module-info.java file.

Module name:

☐ Generate comments (configure templates and default value [here](#))

Don't Create 클릭



완료

4) github에서 프로젝트 가져오기 #3

github에 업로드한 프로젝트에 프로젝트 설정 파일도 들어있다면 (혼자 개발할 때는 그래도 됨), 프로젝트를 생성할 적당한 디렉토리에서 다음 명령을 실행한다.

```
git clone URL
```

위 명령을 실행하면 remote repository가 다운로드 되어서 현재 디렉토리에 생성된다.

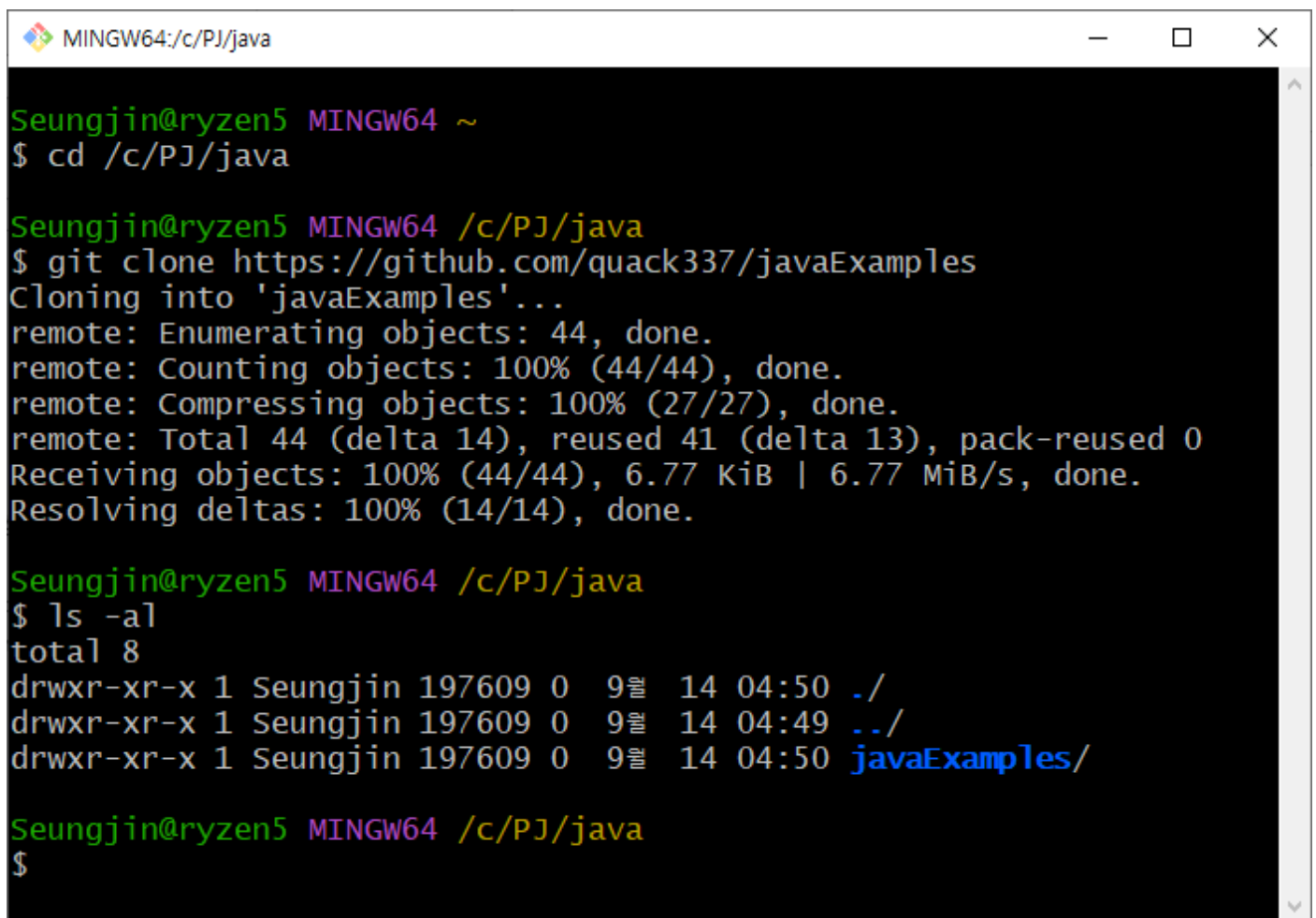
예를 들어 나 혼자 개발하는 개인 프로젝트는 javaExamples에는 remote repository에 프로젝트 설정 파일도 들어있다.
이 remote repository를 다운로드 하자.

먼저 프로젝트를 생성할 적당한 디렉토리로 이동하고

```
cd /c/PJ/java/
```

remote repository를 다운로드 한다.

```
git clone https://github.com/quack337/javaExamples
```



```
MINGW64:/c/PJ/java

Seungjin@ryzen5 MINGW64 ~
$ cd /c/PJ/java

Seungjin@ryzen5 MINGW64 /c/PJ/java
$ git clone https://github.com/quack337/javaExamples
Cloning into 'javaExamples'...
remote: Enumerating objects: 44, done.
remote: Counting objects: 100% (44/44), done.
remote: Compressing objects: 100% (27/27), done.
remote: Total 44 (delta 14), reused 41 (delta 13), pack-reused 0
Receiving objects: 100% (44/44), 6.77 KiB | 6.77 MiB/s, done.
Resolving deltas: 100% (14/14), done.

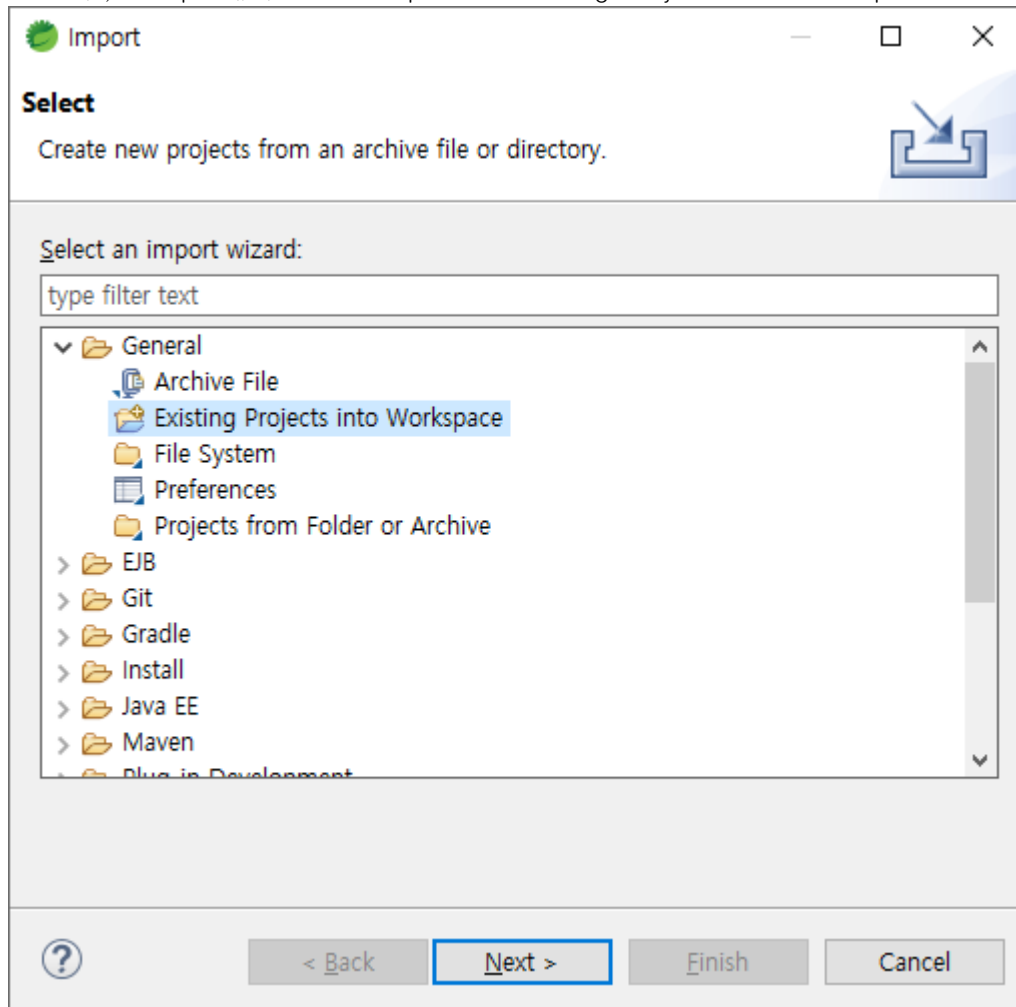
Seungjin@ryzen5 MINGW64 /c/PJ/java
$ ls -al
total 8
drwxr-xr-x 1 Seungjin 197609 0  9월  14  04:50 ./
drwxr-xr-x 1 Seungjin 197609 0  9월  14  04:49 ../
drwxr-xr-x 1 Seungjin 197609 0  9월  14  04:50 javaExamples/

Seungjin@ryzen5 MINGW64 /c/PJ/java
$
```

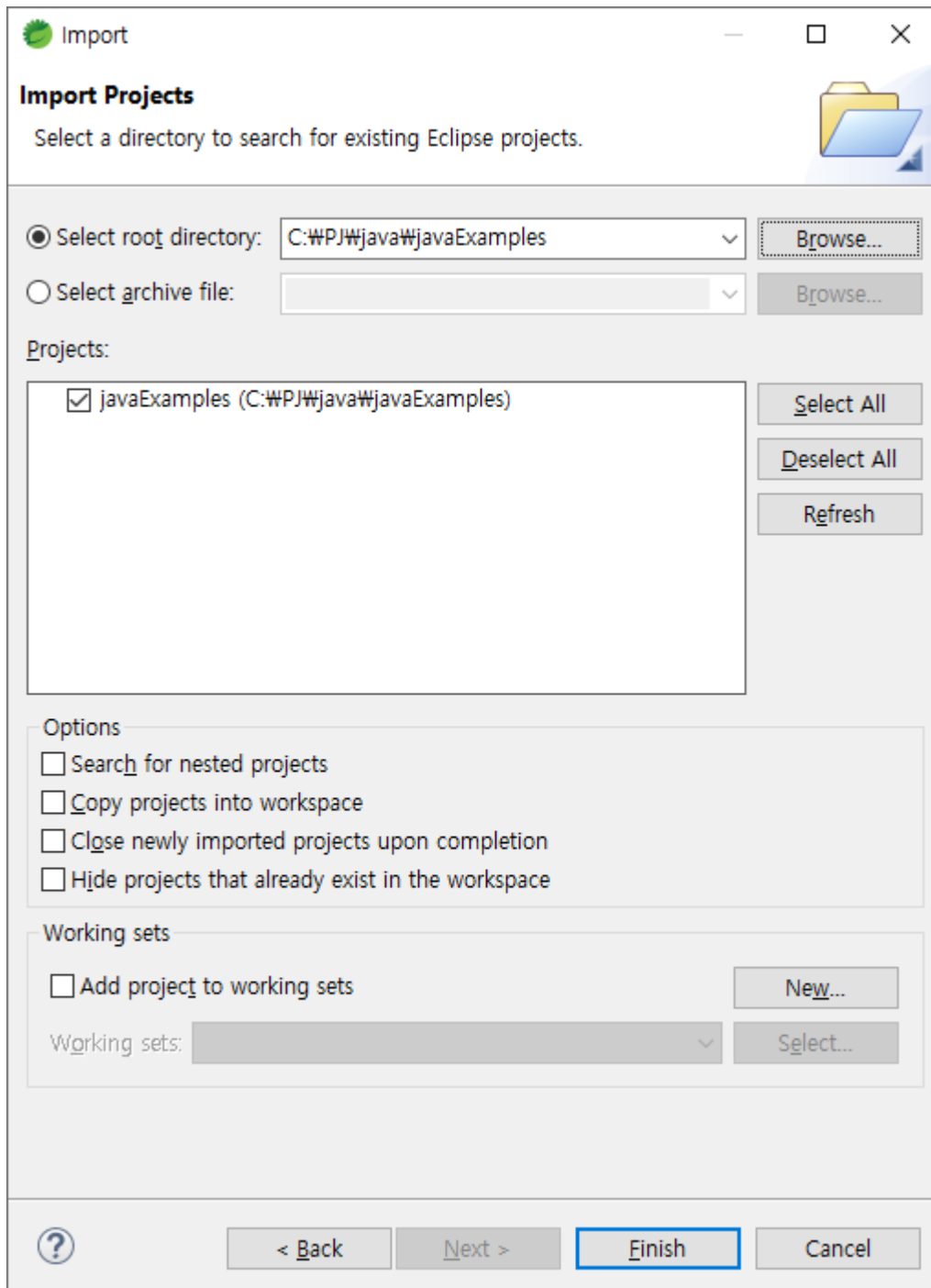
현재 디렉토리는 /c/PJ/java 이고

여기에 javaExamples 프로젝트가 다운로드 되어 생성되었다.

그런 후, eclipse에서 File - Import - Existing Projects into Workspace



Next 버튼을 클릭하고



이 대화상자에서 Browse 버튼을 클릭하여,
git clone 명령으로 생성된 프로젝트 디렉토리를 선택해주고
Finish 버튼을 클릭하면 된다.

5) github에서 프로젝트 가져오기 #4

팀 프로젝트의 경우에 프로젝트 설정 파일은 repository에 등록하면 안된다.

이 경우에는

- (1) eclipse에서 프로젝트를 생성하고, (팀장과 동일한 방법으로 생성)
- (2) 임시 디렉토리에 remote repository를 다운로드 받아서
- (3) 다운로드된 디렉토리나 파일들을, 1)에서 생성한 프로젝트 디렉토리로 옮겨주면 된다.

(1) 프로젝트 생성

The screenshot shows the 'New Java Project' dialog box in Eclipse. The title bar says 'New Java Project'. Below the title bar, it says 'Create a Java Project' and 'Create a Java project in the workspace or in an external location.' There is a folder icon on the right. The 'Project name' field is filled with 'git_test2'. The 'Use default location' checkbox is checked. The 'Location' field is filled with 'C:\WPJ\STS\java\git_test2'. There is a 'Browse...' button next to the location field. Under the 'JRE' section, the 'Use an execution environment JRE:' radio button is selected, and the dropdown menu shows 'JavaSE-11'. There are also options for 'Use a project specific JRE:' (with a dropdown showing 'jre') and 'Use default JRE 'jre' and workspace compiler preferences'. A 'Configure JREs...' link is present. Under the 'Project layout' section, the 'Create separate folders for sources and class files' radio button is selected, with a 'Configure default...' link. Under the 'Working sets' section, the 'Add project to working sets' checkbox is unchecked. There are 'New...' and 'Select...' buttons. Under the 'Module' section, the 'Create module-info.java file' checkbox is checked. At the bottom, there is an information icon and a message: 'The default compiler compliance level for the current workspace is 16. The new project will use a project specific compiler compliance level of 11. Configure...'. The bottom bar has buttons for '< Back', 'Next >', 'Finish' (highlighted with a blue border), and 'Cancel'.

New Java Project

Create a Java Project

Create a Java project in the workspace or in an external location.

Project name:

☒ Use default location

Location:

JRE

☒ Use an execution environment JRE:

☐ Use a project specific JRE:

☐ Use default JRE 'jre' and workspace compiler preferences [Configure JREs...](#)

Project layout

☐ Use project folder as root for sources and class files

☒ Create separate folders for sources and class files [Configure default...](#)

Working sets

☐ Add project to working sets

Working sets:

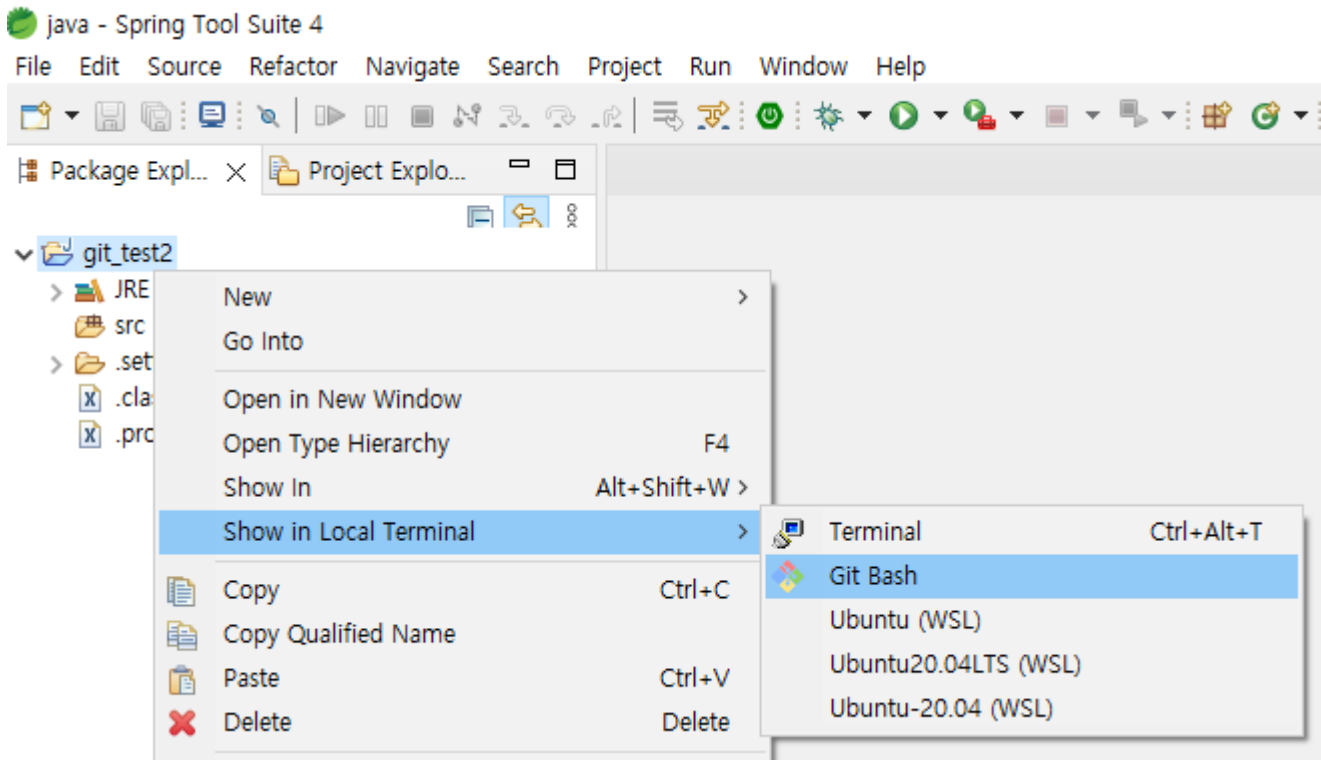
Module

☒ Create module-info.java file

i The default compiler compliance level for the current workspace is 16. The new project will use a project specific compiler compliance level of 11. [Configure...](#)

팀장과 동일한 방법으로 프로젝트를 생성한다.

(2) remote repository 다운로드



git bash 셸을 열자.

프로젝트 디렉토리에 git bash 셸이 열린다.

다음 명령을 실행하자.

```
git clone https://github.com/quack337/git_test2
```

위 명령을 실행하면, 현재 디렉토리 아래에 git_test2 프로젝트 디렉토리가 생성된다.

현재 디렉토리는 `/c/PJ/STS/git_test2` 이다.

이 디렉토리 아래에 remote repository가 다운로드 되어 `/c/PJ/STS/git_test2/git_test2` 디렉토리가 생성되었다.

```
Terminal ×
MINGW64:/c/PJ/STS/git_test2 ×

Seungjin@ryzen5 MINGW64 /c/PJ/STS/git_test2 (main)
$ git clone https://github.com/quack337/git_test2
Cloning into 'git_test2'...
remote: Enumerating objects: 11, done.
remote: Counting objects: 100% (11/11), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 11 (delta 1), reused 11 (delta 1), pack-reused 0
Receiving objects: 100% (11/11), done.
Resolving deltas: 100% (1/1), done.

Seungjin@ryzen5 MINGW64 /c/PJ/STS/git_test2 (main)
$ ls -al
total 14
drwxr-xr-x 1 Seungjin 197609  0  9월 14 05:07 ./
drwxr-xr-x 1 Seungjin 197609  0  9월 14 05:03 ../
-rw-r--r-- 1 Seungjin 197609 386  9월 14 05:03 .classpath
drwxr-xr-x 1 Seungjin 197609  0  9월 14 05:06 .git/
-rw-r--r-- 1 Seungjin 197609 368  9월 14 05:03 .project
drwxr-xr-x 1 Seungjin 197609  0  9월 14 05:03 .settings/
drwxr-xr-x 1 Seungjin 197609  0  9월 14 05:03 bin/
drwxr-xr-x 1 Seungjin 197609  0  9월 14 05:07 git_test2/
drwxr-xr-x 1 Seungjin 197609  0  9월 14 05:03 src/

Seungjin@ryzen5 MINGW64 /c/PJ/STS/git_test2 (main)
$ ls -al git_test2/
total 8
drwxr-xr-x 1 Seungjin 197609 0  9월 14 05:07 ./
drwxr-xr-x 1 Seungjin 197609 0  9월 14 05:07 ../
drwxr-xr-x 1 Seungjin 197609 0  9월 14 05:07 .git/
drwxr-xr-x 1 Seungjin 197609 0  9월 14 05:07 src/

Seungjin@ryzen5 MINGW64 /c/PJ/STS/git_test2 (main)
$
```

```
ls -al git_test2/
```

위 명령은 현재 디렉토리 아래의 git_test2 디렉토리 내용을 보여준다.

- .git local repository 디렉토리
- src 소스코드 디렉토리가 보인다.
- 이 디렉토리들을 현재 디렉토리로 옮기자.

```
mv git_test2/.git git_test2/* .
```

- 위 명령은 현재 디렉토리 아래 git_test2/.git 디렉토리와
현재 디렉토리 아래 git_test2/* 항목들을 (모든 파일이나 디렉토리들을)
현재 디렉토리로 이동(mv)한다.

```

Seungjin@ryzen5 MINGW64 /c/PJ/STS/git_test2
$ mv git_test2/.git git_test2/* .

Seungjin@ryzen5 MINGW64 /c/PJ/STS/git_test2 (main)
$ ls -al
total 14
drwxr-xr-x 1 Seungjin 197609  0  9월 14 05:13 ./
drwxr-xr-x 1 Seungjin 197609  0  9월 14 05:03 ../
-rw-r--r-- 1 Seungjin 197609 386  9월 14 05:03 .classpath
drwxr-xr-x 1 Seungjin 197609  0  9월 14 05:13 .git/
-rw-r--r-- 1 Seungjin 197609 368  9월 14 05:03 .project
drwxr-xr-x 1 Seungjin 197609  0  9월 14 05:03 .settings/
drwxr-xr-x 1 Seungjin 197609  0  9월 14 05:03 bin/
drwxr-xr-x 1 Seungjin 197609  0  9월 14 05:13 git_test2/
drwxr-xr-x 1 Seungjin 197609  0  9월 14 05:13 src/

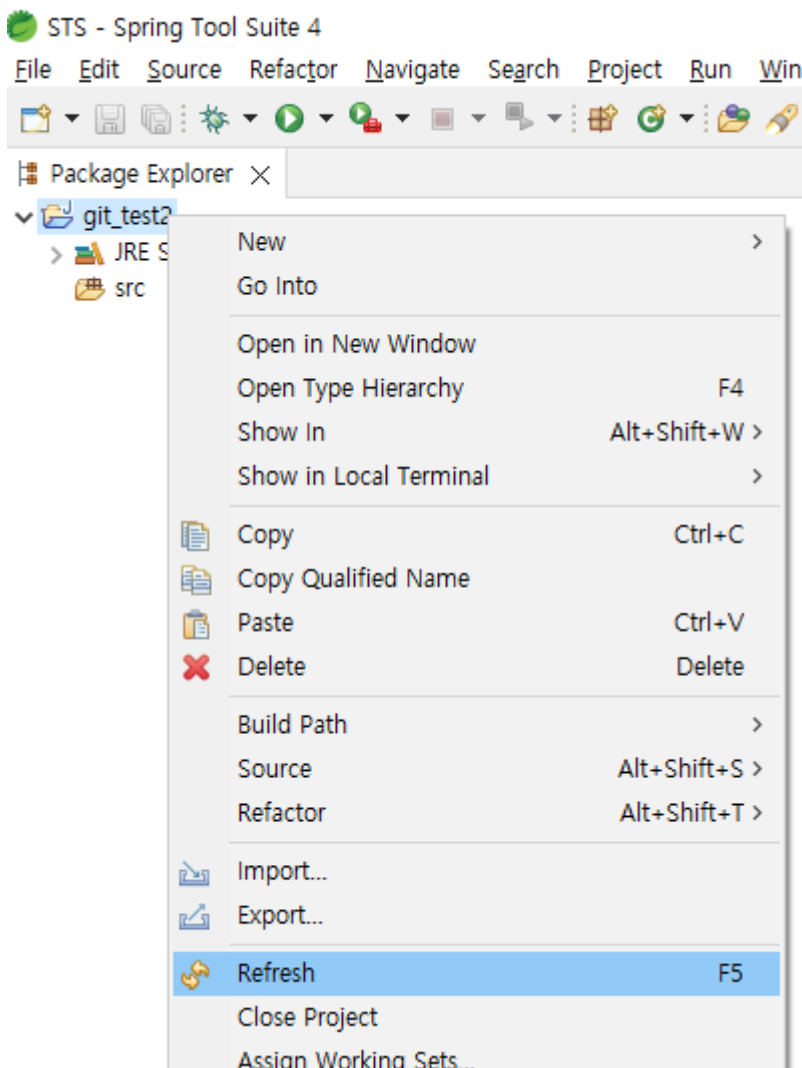
Seungjin@ryzen5 MINGW64 /c/PJ/STS/git_test2 (main)
$ rmdir git_test2

Seungjin@ryzen5 MINGW64 /c/PJ/STS/git_test2 (main)
$ █

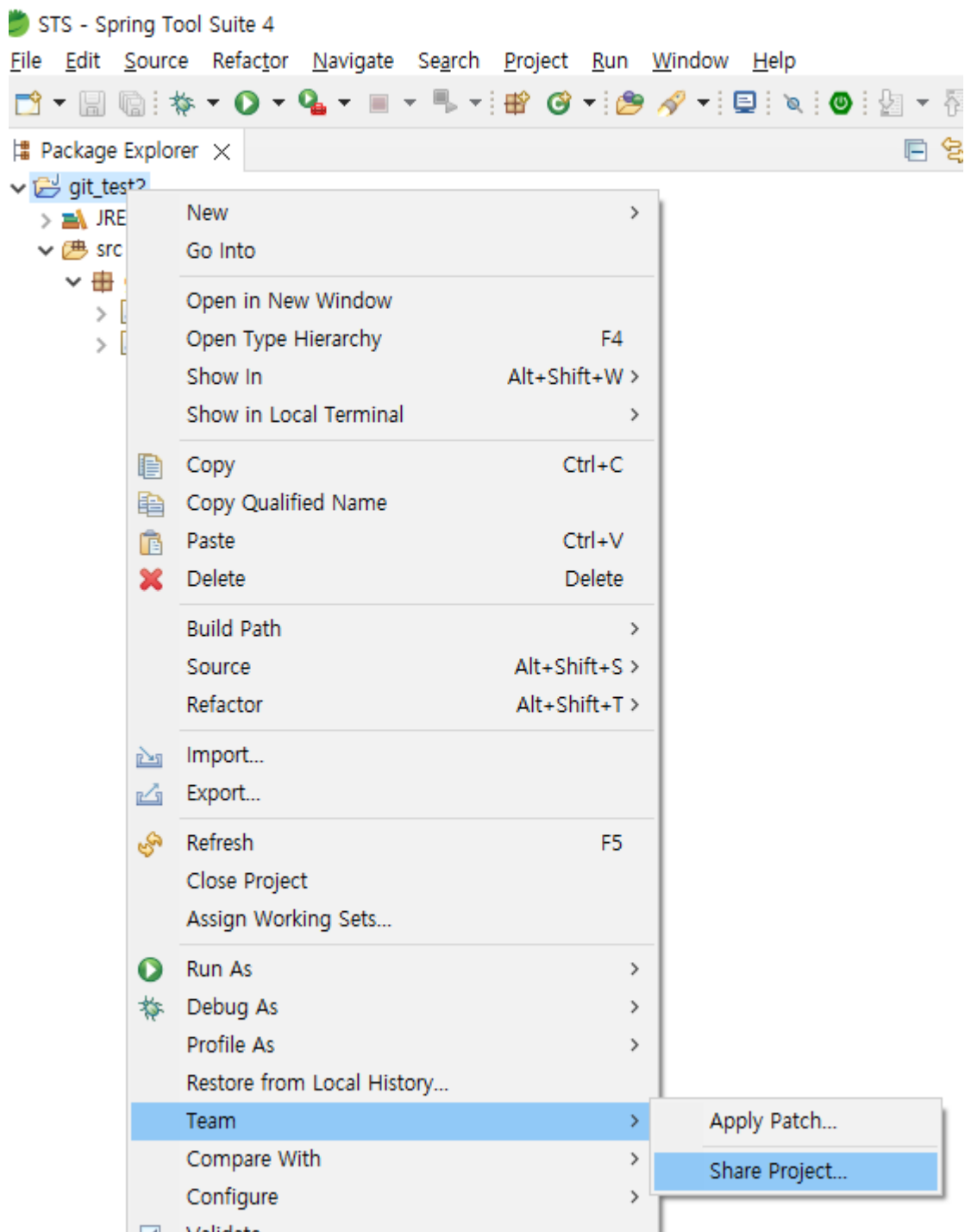
```

```
rmdir git_test2
```

내용을 모두 옮겼으므로, 현재 디렉토리 아래 git_test2 디렉토리를 이제 비었다.
rmdir 명령은 비어있는 디렉토리를 삭제한다. 내용이 있으면 삭제 안된다.



git_test2 프로젝트를 Refresh 하면,
다운로드 되어 옮겨진 src 디렉토리의 내용이 보인다.



Team - Share Project

위 메뉴를 실행하면, .git 로컬 리포지토리가 인식된다.

6. 과제

위 실습 절차를 그대로 따라서 수행하고,
그 과정에서 생성된 git_test2 remote repository URL을 제출하라.

예: `https://github.com/quack337/git_test2.git`
github remote repository URL은 .git 으로 끝난다.